

Abstract. In this paper, we describe a method for solving the Eikonal equation on domains in $\mathbb{R}^2$ or $\mathbb{R}^3$ tessellated by triangles or tetrahedra. Our method generalizes to simplices in higher dimensions, owing to our choice of solution representation under the Jacobi family of orthogonal polynomials on the simplex. First, we describe a spectral decomposition for functions supported on the simplex in terms of their weighted L_2 expansion coefficients under the Jacobi basis. Computing this expansion requires an accurate and efficient quadrature, and we provide a novel optimization routine based on approximate joint-diagonalization of Jacobi matrices for generating such quadratures. Then, we consider differential operators acting on coefficients of a Jacobi expansion. Recent results due to Townsend, Oliver and Vasil enable the construction of sparse differential operators on the standard triangle, interoperating within the Jacobi polynomial family parameter space, and leading to low-storage, banded discrete differential operators. We generalize these results to the tetrahedra (laying the path for generalizing to $d > 3$), and use the resulting differential operators to construct the Eikonal operator on the simplex. Ultimately, we must solve a non-linear PDE, and working in coefficient space implies that our solution variables must be thought of as functions. Hence, we reformulate the Eikonal problem in terms of an infinite-dimensional L_2 minimization problem, which can be discretized with the aforementioned numerical quadrature. As with the README, this is more of a blog, but it's much easier to typeset math here.

Key words. multivariate orthogonal polynomials, spectral methods, partial differential equations, Eikonal equations, simplex

MSC codes. 68Q25, 68R10, 68U05

1. Global representations to local solutions of the Eikonal Equation. Haven't really looked into the viscosity perspective, but..I think I converged on the same idea for solving the PDE. Consider the standard simplex

$$\mathcal{T}^d = \{\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^d \mid x_i \geq 0, 1 - |\mathbf{x}|_1 \geq 0\}$$

The *minimal* time $u(\mathbf{x})$ to reach the boundary $\partial\mathcal{T}^d$ from a point $\mathbf{x} \in \mathcal{T}^d$ is computable through solving the Eikonal equation on $\mathcal{T}^d$:

$$(1.1) \quad \begin{cases} \sum_{i=1}^d (\partial_{x_i} u)^2 = \left(\frac{1}{f}\right)^2, & \mathbf{x} \in \mathcal{T}^d, \\ u = q, & \mathbf{x} \in \partial\mathcal{T}^d, \end{cases}$$

where $f : \overline{\mathcal{T}^d} \rightarrow \mathbb{R}_*^+$ is the speed in the medium, $q : \partial\mathcal{T}^d \rightarrow \mathbb{R}^+$ is the time to leave $\mathcal{T}^d$ from a point on $\partial\mathcal{T}^d$. We seek to represent u under an optimal, finite, weighted L_2 expansion, wherein

$$u(\mathbf{x}) \approx \sum_{j=1}^n u_j P_j(\mathbf{x}) = \mathbf{u} \cdot \mathbf{P}(\mathbf{x}),$$

and $\mathbf{u}$ is the vector in $\mathbb{R}^n$ of coefficients of u under the elements of the basis $\mathbf{P}(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}^n$. Here $\approx$ must be taken in the sense of weighted L_2 convergence of the series to u as $n \rightarrow \infty$.

1.1. Jacobi bases on the Triangle. Let us specialize for $d = 2$. By P_k^n , we denote the k^{th} Jacobi polynomial on the triangle of degree n with parameters (a, b, c) . When the parameters must be specified, we will refer to the same polynomial by $P_{k,n}^{(a,b,c)}$. The weight function is

$$(1.2) \quad \begin{aligned} W^{(a,b,c)}(x, y) &= x^{a-\frac{1}{2}} y^{b-\frac{1}{2}} (1-x-y)^{c-\frac{1}{2}}, \\ w_{(a,b,c)} &= \left(\int_{\mathcal{T}_{\text{ref}}^2} W(x, y) dx dy \right)^{-1} = \frac{\Gamma(|\kappa| + \frac{3}{2})}{\Gamma(a + \frac{1}{2}) \Gamma(b + \frac{1}{2}) \Gamma(c + \frac{1}{2})} \\ w^{(a,b,c)}(\mathbf{x}) &= w_{(a,b,c)} W^{(a,b,c)}(\mathbf{x}) \end{aligned}$$

with $a, b, c > -1/2$ the Jacobi parameters, and $|\kappa| = a + b + c$. The orthonormal polynomials are given in terms of the 1D Jacobi polynomials by

$$(1.3) \quad \begin{aligned} P_k^n(x, y) &= (h_{k,n})^{-1} P_{n-k}^{2k+b+c, a-1/2}(2x-1)(1-x)^k P_k^{c-1/2, b-1/2}\left(\frac{2y}{1-x} - 1\right), \\ (h_{k,n})^2 &= \frac{w_{a,b,c}}{(2n + |\kappa| + \frac{1}{2})(2k + b + c)} \\ &\times \frac{\Gamma(n + k + b + c + 1) \Gamma(n - k + a + \frac{1}{2}) \Gamma(k + b + \frac{1}{2}) \Gamma(k + c + \frac{1}{2})}{(n - k)! k! \Gamma(n + k + |\kappa| + \frac{1}{2}) \Gamma(k + b + c)}, \end{aligned}$$

and they satisfy a mutual orthonormality condition under the weighted inner product:

$$(1.4) \quad \langle P_{j,m}^{(a,b,c)}, P_{k,n}^{(a,b,c)} \rangle_{w^{(a,b,c)}} = \delta_{jk} \delta_{mn}.$$

*Submitted to the editors DATE.

Funding: This work is partially supported by the National Science Foundation under Grant Number 2110886.

†Department of Applied Mathematics, University of Colorado, Boulder, CO

For the 1D Jacobi polynomials, we have convenient explicit evaluations of the polynomials at the endpoints (for reference, see the DLMF by NIST). On the triangle, we have.

- (i) $P_k^n(0, y) = (h_{k,n})^{-1} (-1)^{n-k} \binom{n-k+a-1/2}{n-k} P_k^{c-1/2, b-1/2}(2y-1)$
- (ii) $P_k^n(x, 0) = (h_{k,n})^{-1} P_{n-k}^{2k+b+c, a-1/2}(2x-1)(1-x)^k (-1)^k \binom{k+b-1/2}{k}$
- (iii) $P_k^n(x, 1-x) = (h_{k,n})^{-1} P_{n-k}^{2k+b+c, a-1/2}(2x-1)(1-x)^k \binom{k+c-1/2}{k}$

Moreover, we must derive a limit relation for evaluating the polynomials at $x = 1$, the bottom right corner of the reference triangle¹. That is, we seek

$$\lim_{x \rightarrow 1} P_k^{n, (a, b, c)}(x, y) = (h_{k,n})^{-1} \binom{n+k+b+c}{n-k} \lim_{x \rightarrow 1} (1-x)^k P_k^{c-1/2, b-1/2} \left(\frac{2y}{1-x} - 1 \right),$$

where we used the endpoint formula for the Jacobi polynomials on the interval $[-1, 1]$ to reduce the limit. We proceed by a dominating order argument. The term $(1-x)^k \rightarrow 0$ as $x \rightarrow 1$, while

$$\lim_{x \rightarrow 1} P_k^{c-1/2, b-1/2} \left(\frac{2y}{1-x} - 1 \right) \propto \lim_{x \rightarrow 1} \left(\frac{2y}{1-x} - 1 \right)^k$$

is indeterminate. However, the combined limit can be easily solved as

$$\lim_{x \rightarrow 1} (1-x)^k \left(\frac{2y}{1-x} - 1 \right)^k = (2y)^k,$$

and $(1-x)^k \left(\frac{2y}{1-x} - 1 \right)^l \rightarrow 0$ as $x \rightarrow 1$ for any $l < k$. Hence, the leading order coefficient of $P_k^{c-1/2, b-1/2} \left(\frac{2y}{1-x} - 1 \right)$ is the needed proportionality constant (see DLMF), so that

$$(1.5) \quad \lim_{x \rightarrow 1} P_k^{n, (a, b, c)}(x, y) = (h_{k,n})^{-1} \binom{n+k+b+c}{n-k} \frac{(k+c+b)_k}{2^k k!} (2y)^k,$$

where $(a)_n$ is the Pochhammer symbol.

A given basis polynomial is not only orthonormal to those with different total degree, but also to those others within the same-degree homogeneous subspace. In other words, P_k^n , $0 \leq k \leq n$ forms a basis for the space of homogeneous polynomials of degree n . By considering ordered collections of these subspaces (where ordering is in terms of the degree n of the space), instead of individual polynomials, we can more elegantly reformulate the Eikonal problem into a weak-type form. Let

$$\mathbb{P}_n(x_1, x_2) = (P_0^n(x_1, x_2), \dots, P_n^n(x_1, x_2))^T,$$

and let

$$(1.6) \quad \mathbf{P} = (\mathbb{P}_0^T, \mathbb{P}_1^T, \dots, \mathbb{P}_{n-1}^T)^T.$$

The above notation amounts to a vectorization of the basis for the general space of polynomials in two variables Π_{n-1}^2 defined of $\mathcal{T}^2$. Now, consider the $(n-1)$ order Jacobi polynomial expansion of the solution u under the parameter family (a, b, c) . For $\mathbf{x} \in \mathcal{T}^d$, and $N = \dim \Pi_{n-1}^2$ coefficients $\mathbf{u} \in \mathbb{R}^N$, we can write it as

$$(1.7) \quad u(\mathbf{x}) = \mathbf{P}^{(a, b, c)}(\mathbf{x})^T \mathbf{u}.$$

There exists sparse discrete operators which map $\mathbf{u}$ to the coefficients of the derivative of u under a modified basis, as well as those which promote the basis parameters without taking derivatives:

$$(1.8) \quad \begin{aligned} \partial_{x_1} u(\mathbf{x}) &= \mathbf{P}^{(a+1, b+1, c+1)}(\mathbf{x})^T \mathcal{K}_{(a+1, b, c+1)}^{(a+1, b+1, c+1)} \mathcal{D}_{x_1} \mathbf{u} \\ \partial_{x_2} u(\mathbf{y}) &= \mathbf{P}^{(a+1, b+1, c+1)}(\mathbf{x})^T \mathcal{K}_{(a, b+1, c+1)}^{(a+1, b+1, c+1)} \mathcal{D}_{x_2} \mathbf{u} \end{aligned}$$

Inserting this into the fully constrained Eikonal equation gives

$$(1.9) \quad \begin{aligned} & \left(\mathbf{P}^{(a+1, b+1, c+1)}(\mathbf{x})^T \mathcal{K}_{(a, b, c)}^{(a+1, b+1, c+1)} \mathbf{c}_{\frac{1}{f}} \right)^2 = \\ & \left(\mathbf{P}^{(a+1, b+1, c+1)}(\mathbf{x})^T \mathcal{K}_{(a+1, b, c+1)}^{(a+1, b+1, c+1)} \mathcal{D}_{x_1} \mathbf{u} \right)^2 + \\ & \left(\mathbf{P}^{(a+1, b+1, c+1)}(\mathbf{x})^T \mathcal{K}_{(a, b+1, c+1)}^{(a+1, b+1, c+1)} \mathcal{D}_{x_2} \mathbf{u} \right)^2 = \\ (1.10) \quad & \mathbf{P}^{(a+1, b+1, c+1)}(\mathbf{x})^T (\tilde{\mathcal{D}}_{x_1} \mathbf{u} \mathbf{u}^T \tilde{\mathcal{D}}_{x_1}^T + \tilde{\mathcal{D}}_{x_2} \mathbf{u} \mathbf{u}^T \tilde{\mathcal{D}}_{x_2}^T) \mathbf{P}^{(a+1, b, c+1)}(\mathbf{x}), \end{aligned}$$

where

$$(1.11) \quad \tilde{\mathcal{D}}_{x_1} = \mathcal{K}_{(a+1, b, c+1)}^{(a+1, b+1, c+1)} \mathcal{D}_{x_1}, \quad \tilde{\mathcal{D}}_{x_2} = \mathcal{K}_{(a, b+1, c+1)}^{(a+1, b+1, c+1)} \mathcal{D}_{x_2},$$

¹This is necessary for evaluating the Vandermonde matrix at such points, as they may occur during descent iterations for the quadrature problem

and, the coefficients of $(1/f)$ are represented as $\mathbf{c}_{\frac{1}{f}}$ but promoted to the $(a+1, b+1, c+1)$ Jacobi basis. We let $\tilde{\cdot}$ indicate promotion to $(a+1, b+1, c+1)$ from any other parameter set. The above equality holds for any point $\mathbf{x} \in \mathcal{T}^2$. So, for a solution $\mathbf{u}$, the coefficient operation must be such that:

$$(1.12) \quad (\tilde{\mathcal{D}}_{x_1} \mathbf{u} \mathbf{u}^T \tilde{\mathcal{D}}_{x_1}^T + \tilde{\mathcal{D}}_{x_2} \mathbf{u} \mathbf{u}^T \tilde{\mathcal{D}}_{x_2}^T) = \tilde{\mathbf{c}}_{\frac{1}{f}} \tilde{\mathbf{c}}_{\frac{1}{f}}^T.$$

We seek a coefficient solution $\mathbf{u}$ which represents the solution *globally* in $\mathcal{T}^2$. However, polynomials are differentiable, and some of the problems we are interested in yield non-differentiable solutions. This is true even in the most basic case of $f \equiv 1$, in which u is the signed distance function from $\mathbf{x}$ to $\partial\mathcal{T}^2$, depicted below:

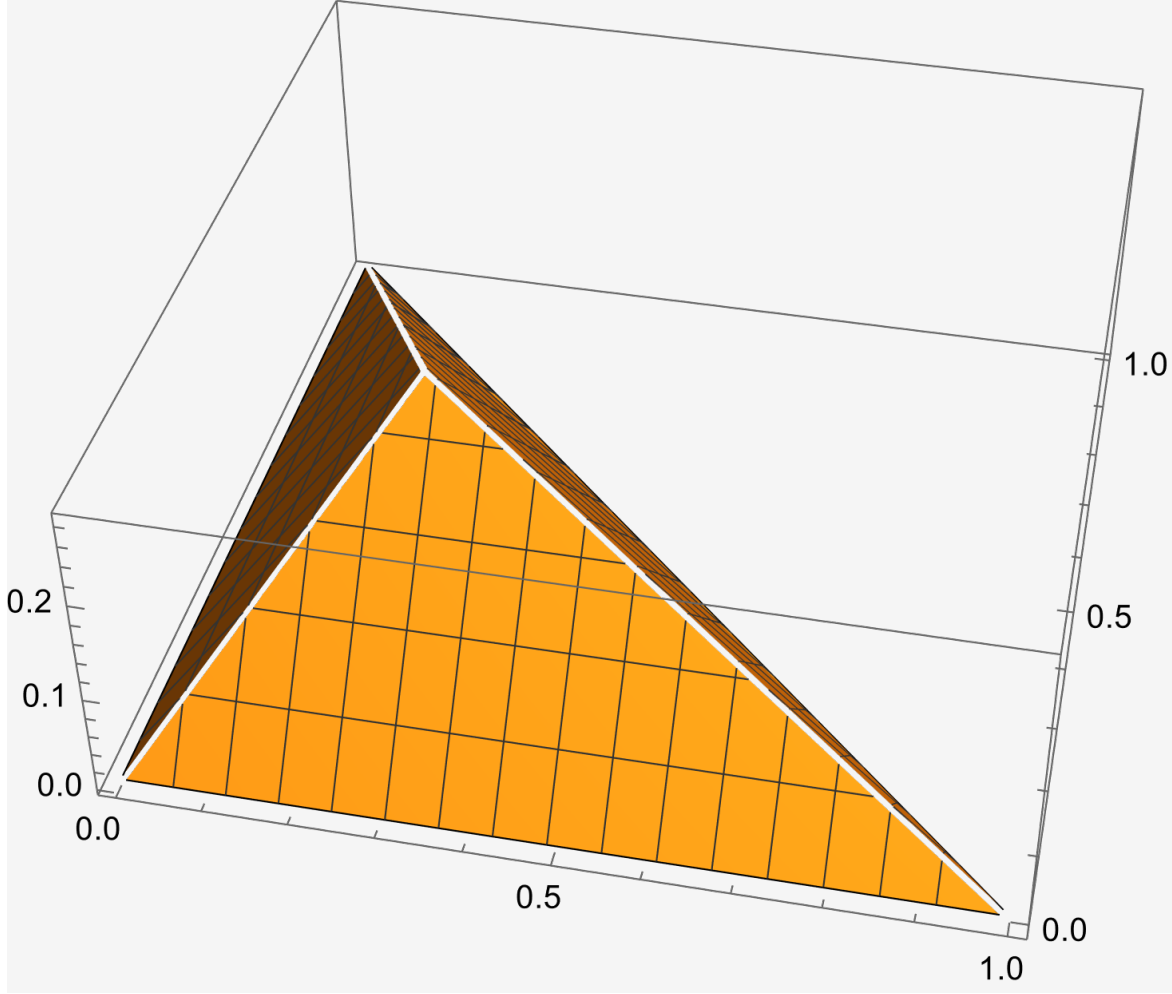


FIG. 1. A simple calculation shows that the minimum distance from $\mathbf{x} \in \mathcal{T}^2$ to the boundary $\partial\mathcal{T}^2$ is given by the minimum of $\left\{x_1, x_2, \left\| \begin{bmatrix} (x_1 - x_2 + 1)/2 \\ (x_2 - x_1 + 1)/2 \end{bmatrix} \right\|_2 \right\}$

We cannot hope to recover a solution $\mathbf{u}$ which satisfies (1.12) to machine precision, as that would imply the global solution has zero residual *pointwise* everywhere. Hence, we must return to (1.9), and consider solving it weakly, in the sense of:

$$(1.13) \quad \begin{aligned} \int_{\mathcal{T}^2} w^{(a+1,b+1,c+1)}(\mathbf{x}) \mathbf{P}^{(a+1,b+1,c+1)}(\mathbf{x})^T \tilde{\mathbf{c}}_{\frac{1}{f}} \tilde{\mathbf{c}}_{\frac{1}{f}}^T \mathbf{P}^{(a+1,b+1,c+1)}(\mathbf{x}) d\mathbf{x} &\approx \|1/f\|_{w^{(a+1,b+1,c+1)}}^2 \\ &\approx \int_{\mathcal{T}^2} w^{(a+1,b+1,c+1)}(\mathbf{x}) \mathbf{P}^{(a+1,b+1,c+1)}(\mathbf{x})^T (\tilde{\mathcal{D}}_{x_1} \mathbf{u} \mathbf{u}^T \tilde{\mathcal{D}}_{x_1}^T + \tilde{\mathcal{D}}_{x_2} \mathbf{u} \mathbf{u}^T \tilde{\mathcal{D}}_{x_2}^T) \mathbf{P}^{(a+1,b+1,c+1)}(\mathbf{x}) d\mathbf{x} \\ &\approx \|\partial_x u\|_{w^{(a+1,b+1,c+1)}}^2 + \|\partial_y u\|_{w^{(a+1,b+1,c+1)}}^2. \end{aligned}$$

Here, the norm is viewed as that of a function in $L^2(\mathcal{T}^2, w^{(a+1,b+1,c+1)})$. Parsing this equation out a bit further (condensing sub/super

scripts), we have

$$\begin{aligned}
& \int_{\mathcal{T}^2} \mathbf{P}(\mathbf{x})^T (\mathbf{u}_x \mathbf{u}_x^T + \mathbf{u}_y \mathbf{u}_y^T) w(\mathbf{x}) \mathbf{P}(\mathbf{x}) d\mathbf{x} \quad (\text{Only } x \text{ part handled below}) \\
&= \int_{\mathcal{T}^2} \mathbf{P}(\mathbf{x})^T \begin{bmatrix} u_{x,1}^2 & u_{x,1}u_{x,2} & \cdots & u_{x,1}u_{x,N} \\ u_{x,2}u_{x,1} & u_{x,2}^2 & \cdots & u_{x,2}u_{x,N} \\ \vdots & \ddots & \ddots & \\ u_{x,N}u_{x,1} & u_{x,N}u_{x,2} & \cdots & u_{x,N}^2 \end{bmatrix} \mathbf{P}(\mathbf{x}) w(\mathbf{x}) d\mathbf{x} \\
&= \int_{\mathcal{T}^2} \begin{bmatrix} P_1(\mathbf{x}) & P_2(\mathbf{x}) & \cdots & P_N(\mathbf{x}) \end{bmatrix} \begin{bmatrix} u_{x,1}^2 & u_{x,1}u_{x,2} & \cdots & u_{x,1}u_{x,N} \\ u_{x,2}u_{x,1} & u_{x,2}^2 & \cdots & u_{x,2}u_{x,N} \\ \vdots & \ddots & \ddots & \\ u_{x,N}u_{x,1} & u_{x,N}u_{x,2} & \cdots & u_{x,N}^2 \end{bmatrix} \begin{bmatrix} P_1(\mathbf{x}) \\ P_2(\mathbf{x}) \\ \vdots \\ P_N(\mathbf{x}) \end{bmatrix} w(\mathbf{x}) d\mathbf{x} \\
&= \int_{\mathcal{T}^2} \begin{bmatrix} P_1(\mathbf{x}) & P_2(\mathbf{x}) & \cdots & P_N(\mathbf{x}) \end{bmatrix} \begin{bmatrix} u_{x,1} \sum_{j=1}^N u_{x,j} P_j(\mathbf{x}) \\ u_{x,2} \sum_{j=1}^N u_{x,j} P_j(\mathbf{x}) \\ \vdots \\ u_{x,N} \sum_{j=1}^N u_{x,j} P_j(\mathbf{x}) \end{bmatrix} w(\mathbf{x}) d\mathbf{x} \\
&= \int_{\mathcal{T}^2} w(\mathbf{x}) \sum_{k=1}^N P_k(\mathbf{x}) u_{x,k} \sum_{j=1}^N u_{x,j} P_j(\mathbf{x}) d\mathbf{x} = \int_{\mathcal{T}^2} w(\mathbf{x}) \sum_{k=1}^N \sum_{j=1}^N u_{x,k} u_{x,j} P_k(\mathbf{x}) P_j(\mathbf{x}) d\mathbf{x} \\
&= \int_{\mathcal{T}^2} \sum_{k=1}^N u_{x,k}^2 P_k^2(\mathbf{x}) w(\mathbf{x}) d\mathbf{x} = \sum_{k=1}^N u_{x,k}^2 \int_{\mathcal{T}^2} P_k^2(\mathbf{x}) dw(\mathbf{x}) \quad (\text{now we add back in } y \text{ part}) \\
&= \sum_{k=1}^N u_{x,k}^2 + \sum_{k=1}^N u_{y,k}^2 \\
&= \mathbf{u}^T (\tilde{\mathcal{D}}_x^T \tilde{\mathcal{D}}_x + \tilde{\mathcal{D}}_y^T \tilde{\mathcal{D}}_y) \mathbf{u} = \tilde{\mathbf{c}}_{\frac{1}{f}}^T \tilde{\mathbf{c}}_{\frac{1}{f}},
\end{aligned}$$

where we've used the mutual orthonormality relations, which persist even within a subspace of homogeneous polynomials of degree m , to eliminate all off-diagonal terms in the sum. Note, not all orthonormal bases are created equal, and we normally would have to retain many outer product terms in the sum because orthonormality between vectors of the same subspace is not a given.

Thus, we want to solve the following quadratic program with equality constraints:

$$(1.14) \quad \begin{aligned} \min_{\mathbf{u} \in \mathbb{R}^N} F(\mathbf{u}) &= \mathbf{u}^T G \mathbf{u} - \tilde{\mathbf{c}}_{\frac{1}{f}}^T \tilde{\mathbf{c}}_{\frac{1}{f}} \\ \text{subject to} \quad \mathbf{u} \cdot \mathbf{P}(\mathbf{x}) &= 0, \quad \mathbf{x} \in \partial\Omega, \end{aligned}$$

where $G = \tilde{\mathcal{D}}_x^T \tilde{\mathcal{D}}_x + \tilde{\mathcal{D}}_y^T \tilde{\mathcal{D}}_y \in \mathbb{R}^{N \times N}$ is symmetric and positive semi-definite ². The gradient and Hessian of F are given by

$$(1.15) \quad \nabla_{\mathbf{u}} F(\mathbf{u}) = 2G\mathbf{u},$$

$$(1.16) \quad \nabla_{\mathbf{u}}^2 F(\mathbf{u}) = 2G,$$

and we still need to consider the constraint equation. We take the approach of eliminating the constraints by solving a the minimization problem on auxiliary variables. That is, suppose $\mathbf{P}(\mathbf{x}) \in \mathbb{R}^{M \times N}$, $M > N$, has rank p . We can compute the *full SVD* of $\mathbf{P}(\mathbf{x}) = U\Sigma V^T$. Then, the last $n - p$ rows of V^T form a basis for the $\mathcal{N}(\mathbf{P}(\mathbf{x}))$. That is, $V^T = \begin{bmatrix} V_p^T \\ V_{n-p}^T \end{bmatrix}$, and any solution $\mathbf{u}$ satisfying the equality constraints can be expressed as

$$(1.17) \quad \mathbf{u} = V_{n-p} \mathbf{u}_{\text{eq}},$$

Hence, we can solve

$$(1.18) \quad \min_{\mathbf{u}_{\text{eq}} \in \mathbb{R}^N} F(\mathbf{u}_{\text{eq}}) = \mathbf{u}_{\text{eq}}^T V_{n-p}^T G V_{n-p} \mathbf{u}_{\text{eq}} - \tilde{\mathbf{c}}_{\frac{1}{f}}^T \tilde{\mathbf{c}}_{\frac{1}{f}}$$

and finally find $\mathbf{u}$ by computing as in (1.17). Note, the gradient and Hessian of the auxiliary problem are given by

$$(1.19) \quad \nabla_{\mathbf{u}_{\text{eq}}} F(\mathbf{u}_{\text{eq}}) = 2V_{n-p}^T G V_{n-p} \mathbf{u}_{\text{eq}},$$

$$(1.20) \quad \nabla_{\mathbf{u}}^2 F(\mathbf{u}) = 2V_{n-p}^T G V_{n-p}.$$

Note, we can find p by the number of singular values above some small threshold that we choose (near $\varepsilon_{\text{mach}}$) so that it represents numerical rank. There is a ton of material in chapter 12 and 16 of the Nocedal/Wright optimization book. I believe I can prove properties on minimizers to (1.14) (though unnecessary) and maybe use a direct method therein. There still remains the problem of initialization..perhaps we just initialize from the null space, or solve a linearized problem?

It turns out that this squared version of the Eikonal problem is not coercive. That is, there is no associated coercive bilinear form, which we need if we want to initialize the method with something other than the true solution! The elegance of a polynomial identity in coefficient space is, alas, forgone in practice. Instead, we turn to a regularized version of the un-squared Eikonal problem, and try our hand at a modal finite element method.

²Note, $\forall \mathbf{x} \in \mathbb{R}_*^{n+}$, $\mathbf{x}^T G \mathbf{x} = \mathbf{x}^T \tilde{\mathcal{D}}_x^T \tilde{\mathcal{D}}_x \mathbf{x} + \mathbf{x}^T \tilde{\mathcal{D}}_y^T \tilde{\mathcal{D}}_y \mathbf{x} = \|\tilde{\mathcal{D}}_x \mathbf{x}\|^2 + \|\tilde{\mathcal{D}}_y \mathbf{x}\|^2 \geq 0$. We have equality only for any $\mathbf{x} = \alpha \mathbf{e}_1$, as derivatives of the constant polynomial are 0.

1.2. Regularized Eikonal Problem. Here, we consider the regularized Eikonal problem

$$(1.21) \quad \begin{aligned} |\nabla u| - \left(\frac{1}{f}\right) &= \xi \nabla^2 u, \quad x \in \Omega \\ u &= g, \quad x \in \partial\Omega \end{aligned}$$

which we discretize with a *modal* finite-element method. That is, let $\Omega = \mathcal{T} = \bigcup_{k=1}^{N_T} T_k$ be tessellated by triangles T_k . Let A_k map points $\mathbf{x} \in T_k$ to points $\mathbf{r} \in T_{\text{ref}}$ the reference triangle. For $\mathbf{x} \in T_k$, we represent the solution

$$u^k(\mathbf{x}) = \sum_{j=1}^M u_j^k \phi_j(A_k^{-1}\mathbf{x}),$$

where u_j^k are the Jacobi polynomial coefficients associated to Jacobi basis function ϕ_j . Each triangle uses the same basis functions with mapped argument, and we stack the M unknown coefficients for each triangle as

$$\mathbf{U} = \begin{bmatrix} \mathbf{u}^1 \\ \vdots \\ \mathbf{u}^{N_T} \end{bmatrix} \in \mathbb{R}^{MN_T}.$$

We enforce Dirichlet conditions on all boundary edges, and inter-element continuity on all interior edges. Let

$$\mathcal{E}_{\text{dirch}} = \{\partial T_i \cap \partial\Omega\} = \{\varepsilon_{\text{dirch}}^i\}, \quad \mathcal{E}_{\text{int}} = \{\varepsilon_{\text{int}}^i\}$$

be the set of boundary and interior edges in the tessellation $\mathcal{T}$. We have the associated index maps between T_j and boundary edge index i , and interior edge index i with triangles j, k :

$$b(i) = \begin{cases} j, & \varepsilon_{\text{dirch}}^i \cap \partial T_j \neq \emptyset \\ \emptyset, & \text{else} \end{cases}, \quad t(i) = (t_1(i), t_2(i)) = \begin{cases} (j, k), & (\varepsilon_{\text{int}}^i \cap \partial T_j \neq \emptyset) \wedge (\varepsilon_{\text{int}}^i \cap \partial T_k \neq \emptyset) \end{cases}.$$

Note, the map t is always non-empty, as every interior edge is shared by two triangles (**probably have to reformulate notation for this map**). Now, we proceed with the normal weak formulation of (1.21), in which we evaluate the entries of the block diagonal stiffness matrix, nonlinear functional and load vector, $\mathcal{K} \in \mathbb{R}^{MN_T \times MN_T}$ (each of the N_T blocks is $M \times M$), $\mathbf{N}(\mathbf{U}) \in \mathbb{R}^{MN_T}$, $\mathbf{F} \in \mathbb{R}^{MN_T}$, with a Gaussian-like quadrature rule consisting of abscissa and weights $\{\mathbf{r}_j, w_j\}$ (see Section 2) on T_{ref} . For triangle T_k , we have for $i, j = 1, \dots, M$, $k = 1, \dots, N_T$,

$$(1.22) \quad \mathcal{K}_{ij}^k = \int_{T_{\text{ref}}} (A_k^{-1} \nabla \phi_i) \cdot (A_k^{-1} \nabla \phi_j) |A_k| d\mathbf{r}, \quad \mathcal{K} = \begin{bmatrix} \mathcal{K}^1 & & \\ & \ddots & \\ & & \mathcal{K}^{N_T} \end{bmatrix}$$

$$(1.23) \quad \mathcal{N}(\mathbf{u}^k)_i = \int_{T_{\text{ref}}} \phi_i |A_k| \sqrt{\sum_{l=1}^M \sum_{p=1}^M u_l^k u_p^k (A_k^{-1} \nabla \phi_l) \cdot (A_k^{-1} \nabla \phi_p)} d\mathbf{r}, \quad \mathbf{N}(\mathbf{U}) = \begin{bmatrix} \mathcal{N}(\mathbf{u}^1)_1 \\ \vdots \\ \mathcal{N}(\mathbf{u}^{N_T})_M \end{bmatrix}$$

$$(1.24) \quad \mathcal{F}_i^k = \int_{T_{\text{ref}}} \frac{\phi_i}{f} |A_k| d\mathbf{r}, \quad \mathbf{F} = \begin{bmatrix} \mathcal{F}_1^1 \\ \vdots \\ \mathcal{F}_M^{N_T} \end{bmatrix}$$

We use the derivative maps, as in (1.8), to compute the gradients of the basis functions appearing in (1.22)-(1.23). That is,

$$\partial_x \phi_j^{(a,b,c)}(\mathbf{r}) = \partial_x \phi^{(a,b,c)}(\mathbf{r})^T \mathbf{e}_j = \phi^{a+1,b,c+1}(\mathbf{r}) \mathcal{D}_x \mathbf{e}_j,$$

and similar for $\partial_y \phi_j^{(a,b,c)}$. At this point, we should note that our basis functions do not encode the boundary conditions, and are non-conforming between elements. Hence, we enforce Dirichlet boundary conditions and inter-element continuity using Lagrange multipliers. We can do this in two ways - weakly or pointwise. In the weak constraint enforcement, we seek a solution $\mathbf{U}$ which lies in the null-space of a Lagrange multiplier function space, defined as the span of the edge-restricted basis functions. In the point-wise enforcement, we seek a solution which lies in the null-space of a finite-dimensional Lagrange multiplier vector space, which constrains the solution at finitely many points. We take the latter approach. In fact, if we use the same quadrature points on the edges for the strong enforcement as in the quadrature rules for the weak enforcement, numerical satisfaction of the point-wise constraints implies that we satisfy the weak constraints as well.

Consider n_{leg} Legendre points $\{r_j\}$ in $[0, 1]$. Each edge of the reference triangle can be parameterized with this set of points.

- The left edge is $\{\mathbf{x}^l\} = \{(0, r_j)\}$
- The bottom edge is $\{\mathbf{x}^b\} = \{(r_j, 0)\}$
- The hypotenuse edge is $\{\mathbf{x}^h\} = \{(r_j, 1 - r_j)\}$

Say Dirichlet edge $\varepsilon_{\text{dirch}}^i$ is discretized with Legendre points $\mathbf{x}^e$ (it could be any of the edges in the reference). Then, we want

$$(1.25) \quad \phi(\mathbf{x}^e)^T \mathbf{u}^{b(i)} = 0 \text{ (or } g(\mathbf{x}^e) \text{ if condition is non-homogeneous).}$$

Similarly, for shared edge $\varepsilon_{\text{int}}^i$, we require

$$(1.26) \quad \phi(\mathbf{x}^e)^T \mathbf{u}^{t_1(i)} - \phi(\mathbf{x}^e)^T \mathbf{u}^{t_2(i)} = 0.$$

In the actual assembly procedure, care must be taken to ensure these evaluations occur as written. That is, a Dirichlet edge in physical space may be flipped in the reference triangle. A shared edge in physical space may belong to different edges in the sharing triangles when mapped to reference space, and the parametrization may be flipped as well. Hence, we use edge alignment detection procedures to ensure the basis evaluations are aligned. For Dirichlet edges, this is as simple as mapping the edge vertices to the reference triangle, and checking if the mapped vertices are flipped (eg. edge is mapped to (1,0)-(0,0) is bottom flipped). If they are, we flip the rows of $\phi(\mathbf{x}^e)^T$. For shared edges, we detect if one edge is flipped relative to the other by mapping the quadrature points using either triangles' A_j, A_k incidence matrices, and doing the same for the flipped quadrature points for one of the triangles. If the norm of the difference in physical space between the flipped quadrature points of T_k and the un-flipped quadrature points of T_j is $\mathcal{O}(\varepsilon_{\text{mach}})$, then the basis of T_k must be flipped.

Ultimately, we assemble two matrices to enforce these constraints. Let $n_{\text{bnd}} = |\mathcal{E}_{\text{dirch}}|$ and $n_{\text{int}} = |\mathcal{E}_{\text{int}}|$. Then, we define

$$B^{\text{dirch}} \in \mathbb{R}^{n_{\text{bnd}} n_{\text{leg}} \times M N_T}, \quad B^{\text{int}} \in \mathbb{R}^{n_{\text{int}} n_{\text{leg}} \times M N_T}$$

which are both block-sparse. B^{dirch} has n_{bnd} row blocks, one for each boundary edge, and each is of size $n_{\text{leg}} \times M N_T$ with only $n_{\text{leg}} \times M$ non zeros. B^{int} has n_{int} row blocks, on for each interior edge, and each is of size $n_{\text{int}} \times M N_T$, with two separate sub-blocks of size $n_{\text{leg}} \times M$. The specific structure of these matrices depends entirely on the geometry and tessellation, though we can specify how a given row block of either matrix is computed. For boundary edge i ,

$$B_{k, (b(i)-1)M+1:b(i)M}^{\text{dirch}, i} = \phi(\mathbf{x}_k^e)^T,$$

while for shared edge i ,

$$B_{k, (t_1(i)-1)M+1:t_1(i)M}^{\text{int}, i} = \phi(\mathbf{x}_k^e)^T, \quad B_{k, (t_2(i)-1)M+1:t_2(i)M}^{\text{int}, i} = -\phi(\mathbf{x}_k^e)^T$$

For clarity, we visualize the structure of these matrices for a unit-square domain tessellated with 18 triangles and an order 10 discretization.

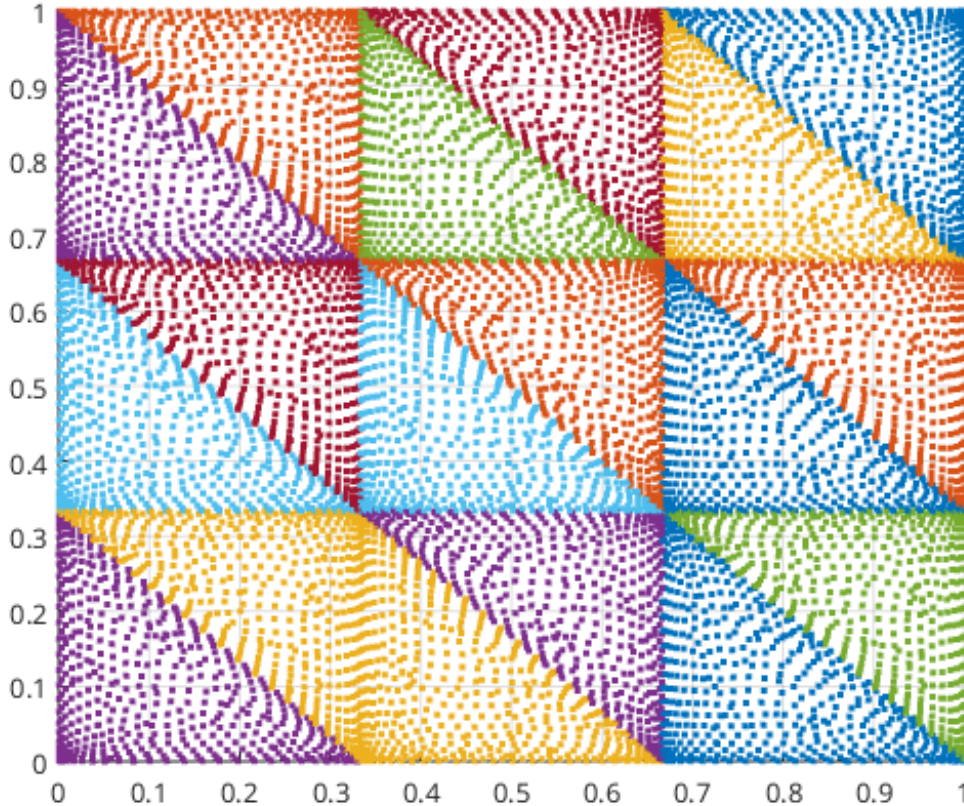


FIG. 2. $[0, 1]^2$ tessellated with 18 triangles, showing mapped quadrature points in each triangle.

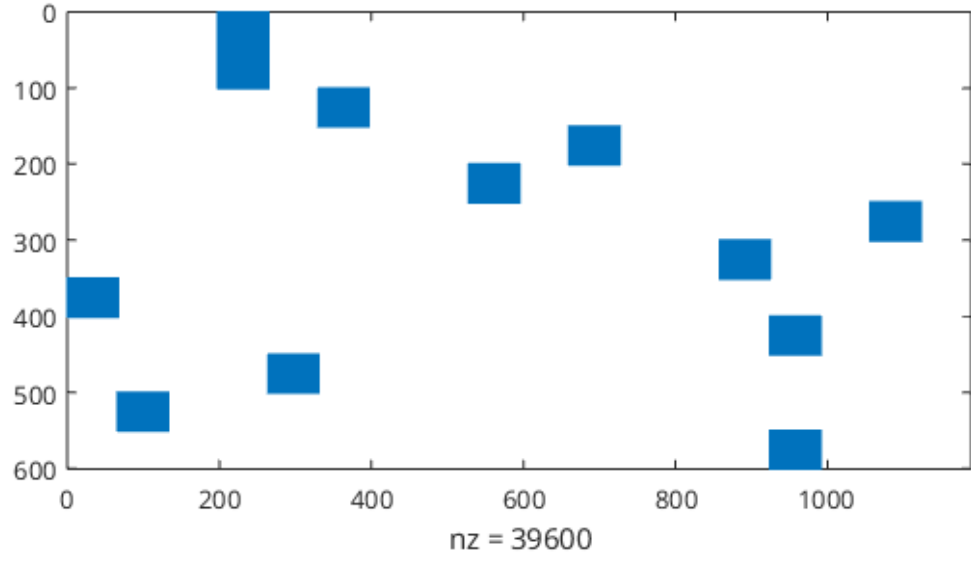


FIG. 3. Structure of B^{dirch} . $M = 66, N_T = 18, n_{bnd} = 12, n_{leg} = 50$

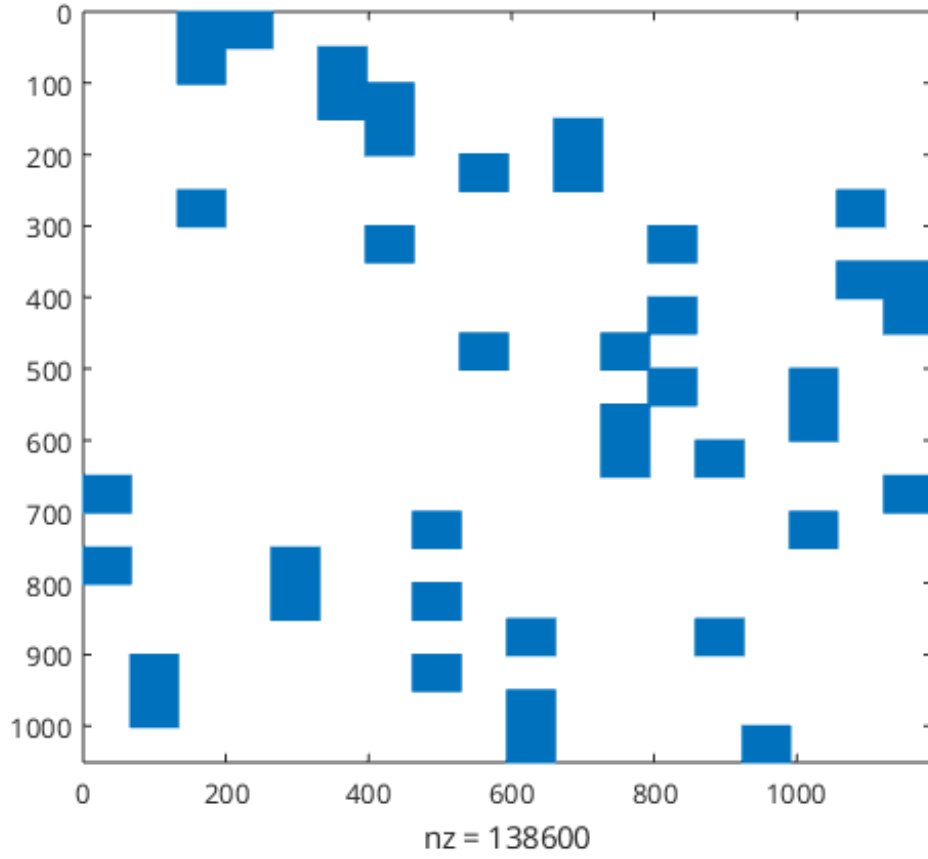


FIG. 4. Structure of B^{int} . $M = 66, N_T = 18, n_{int} = 21, n_{leg} = 50$

The structure we described earlier is evident from these figures. In particular, each block row of B^{int} has two non-zero blocks of size $n_{\text{leg}} \times M$, and each block row of B^{dirch} has only one non-zero block of size $n_{\text{leg}} \times M$. Also note, the bottom-left and upper-right triangles in the mesh both have two boundary edges, and these manifest as block columns in B^{dirch} with two non-zero blocks of size $n_{\text{leg}} \times M$ (see block columns 4 and 15).

At this point, we have not addressed the rank deficiency of the combined matrix

$$B = \begin{bmatrix} B^{\text{int}} \\ B^{\text{dirch}} \end{bmatrix}.$$

There are row redundancies in the constraint equations, both within the Dirichlet and inter-element matrices, as well as between them. To eliminate these redundancies and arrive at a full-rank constraint system, we employ an interpolatory decomposition to select only the independent rows of B , described below.

- Use a rank-revealing QR factorization to get pivot information, providing the independent rows of B . Arrange the pivot information as a vector P so that $B^T(:, P) = QR$. Let $n_\lambda = \text{rank}(B)$.
- Construct $B_{\text{id}} = B(P(1:n_\lambda, :))$. This new matrix has full rank.
- In the case of non-zero Dirichlet conditions g , we must carefully select the correct elements of the assembled vector G , according to the permutation vector P , as in

$$\mathbf{G}_{\text{bc}} = \begin{bmatrix} \mathbf{O}_{n_{\text{leg}} n_{\text{int}}} \\ G \end{bmatrix}, \quad \mathbf{G}_{\text{bc}} \rightarrow \mathbf{G}_{\text{bc}}(P(1:n_\lambda))$$

Now, we can specify the fully discretized variational formulation. We seek $\mathbf{U} \in \mathbb{R}^{MN_T}$, $\mathbf{\Lambda} \in \mathbb{R}^{n_\lambda}$ such that

$$(1.27) \quad \begin{aligned} \mathbf{N}(\mathbf{U}) + \xi \mathcal{K} \mathbf{U} - \mathbf{F} + B_{\text{id}}^T \mathbf{\Lambda} &= 0 \\ B_{\text{id}} \mathbf{U} &= \mathbf{G}_{\text{bc}} \quad (\equiv 0 \text{ for the typical problem}) \end{aligned}$$

Here, we have a constrained non-linear system of equations, and there are many ways in which we can attempt to solve it. We define the residual of the system as

$$(1.28) \quad \mathbf{R}(\mathbf{U}, \mathbf{\Lambda}) = \begin{bmatrix} \mathbf{N}(\mathbf{U}) + \xi \mathcal{K} \mathbf{U} - \mathbf{F} + B_{\text{id}}^T \mathbf{\Lambda} \\ B_{\text{id}} \mathbf{U} - \mathbf{G}_{\text{bc}} \end{bmatrix}.$$

Our goal is to drive this residual to 0.

1.2.1. Infinite-dimensional Newton iteration. The first approach falls under the family of infinite-dimensional Newton steps. That is, we start with an initial guess, typically a solution to the Poisson problem $\nabla^2 u = 1/f$ with the same boundary and inter-element conditions. Then, we seek a descent direction $\delta \begin{bmatrix} \mathbf{U} \\ \mathbf{\Lambda} \end{bmatrix}$ which will reduce the residual $\mathbf{R}(\mathbf{U}, \mathbf{\Lambda})$, update the current solution, and iterate until we achieve some tolerance for the residual. If we view the weak form of (1.21) as corresponding to the first variation of an energy functional $\mathcal{L}$ (whether it does in the right sense is unclear - this is an inverse problem of calculus of variations), then driving the residual to 0 is akin to satisfying the necessary condition for a minimizer of the functional ($\delta \mathcal{L}(u, v) = 0$). Naturally, a strong descent direction can be found by quadratic approximation of the functional in perturbations of u by εw belonging to the same space as v , which requires its second variation, or, in other words, the first variation of the weak form. For triangle T_k , this is

$$(1.29) \quad \mathcal{H}_{ij}(\mathbf{u}^k) = \int_{T_{\text{ref}}} \phi_j |A_k| (A_k^{-1} \nabla \phi_i) \cdot \frac{\left(\sum_{l=1}^M u_l^k (A_k^{-1} \nabla \phi_l) \right)}{\sum_{l=1}^M \sum_{p=1}^M u_l^k u_p^k (A_k^{-1} \nabla \phi_l) \cdot (A_k^{-1} \nabla \phi_p)} d\mathbf{r} + \xi \mathcal{K}_{ij}$$

Then, a Newton iteration which respects the boundary and inter-element continuity conditions at every step and terminates when the residual meets some tolerance `tol` is:

- (i) Initialize a solution $\begin{bmatrix} \mathbf{U}^{(0)} \\ \mathbf{\Lambda}^{(0)} \end{bmatrix}$ by solving the discretized Poisson problem with constraints

$$\begin{bmatrix} \mathcal{K} & B_{\text{id}}^T \\ B_{\text{id}} & 0 \end{bmatrix} \begin{bmatrix} \mathbf{U}^{(0)} \\ \mathbf{\Lambda}^{(0)} \end{bmatrix} = \begin{bmatrix} \mathbf{F} \\ \mathbf{G}_{\text{bc}} \end{bmatrix}.$$

- (ii) Let $\begin{bmatrix} \mathbf{U} \\ \mathbf{\Lambda} \end{bmatrix} = \begin{bmatrix} \mathbf{U}^{(0)} \\ \mathbf{\Lambda}^{(0)} \end{bmatrix}$.

- (iii) If $\|\mathbf{R}(\mathbf{U}, \mathbf{\Lambda})\| > \text{tol}$, solve for descent direction $\begin{bmatrix} \delta \mathbf{U} \\ \delta \mathbf{\Lambda} \end{bmatrix}$ through

$$\begin{bmatrix} \mathcal{H}^T & B_{\text{id}}^T \\ B_{\text{id}} & 0 \end{bmatrix} \begin{bmatrix} \delta \mathbf{U} \\ \delta \mathbf{\Lambda} \end{bmatrix} = -\mathbf{R}(\mathbf{U}, \mathbf{\Lambda})$$

The constraints ensure that the computed descent direction still lies in the correct, constrained solution space.

(iv) Update the solution with an optional step length $0 < \alpha < 1$.

$$\begin{bmatrix} U \\ \Lambda \end{bmatrix} \rightarrow \begin{bmatrix} U \\ \Lambda \end{bmatrix} + \alpha \begin{bmatrix} \delta U \\ \delta \Lambda \end{bmatrix}$$

(v) Repeat from step (iii) until the residual norm is less than `tol`.

This approach works for viscosity parameter $\xi \sim \mathcal{O}(10^{-2})$, but is rather unstable for smaller ξ . This holds true even if we use homotopic continuation methods, where we have a loop wrapping around the Newton method which starts with $\xi = 0.1$, decreases ξ when the residual tolerance is met, and repeats with the solution for the previous ξ as the initial guess for the Newton iteration. I found that as $\xi \rightarrow 0$, the magnitude of solution modes across the spectrum “lifts” up. This means that $\xi \nabla^2 u$ serves as a smoothing operator on high frequency solution components, and decreasing ξ too much results in a degenerate weak form. The Newton step will have many null components in modal directions that don’t affect the residual, and the new direction drifts into moving the higher-energy non-physical directions (higher degree modes). This takes us away from the correct viscosity limit.

1.2.2. Pseudo-time relaxation with an IMEX method. The second approach is one in which we reduce the residual by casting the problem as a system of first-order ordinary differential algebraic equations (DAEs) with a fictitious time parameter τ . We write

$$(1.30) \quad \partial_\tau U = -R(U, \Lambda),$$

and discretize this DAE system in time with common techniques. The first we try is an implicit-explicit discretization, where we handle the non-linear term explicitly, and the stiff linear term implicitly. That is,

$$(1.31) \quad \begin{aligned} \frac{U^{n+1} - U^n}{\Delta\tau} &= -(N(U^n) + \xi K U^{n+1} - F) \\ \Rightarrow (I + \xi \Delta\tau K) U^{n+1} &= U^n - \Delta\tau (N(U^n) - F) \end{aligned}$$

We want to enforce the constraints at each time step, so we solve the augmented system

$$(1.32) \quad \begin{bmatrix} I + \xi \Delta\tau K & B_{\text{id}}^T \\ B_{\text{id}} & 0 \end{bmatrix} \begin{bmatrix} U^{n+1} \\ \Lambda^{n+1} \end{bmatrix} = \begin{bmatrix} U^n - \Delta\tau (N(U^n) - F) \\ G_{\text{bc}} \end{bmatrix}$$

So far, this method is the fastest, and most accurate in terms of converging to a reasonable solution for moderate ξ . A plot of the solution is depicted below for the unit square. Note, ξ has the effect of preserving the behavior of the time to boundary, but either reducing or increasing the maximum value (which occurs at the medial point $(1/2, 1/2)$.)

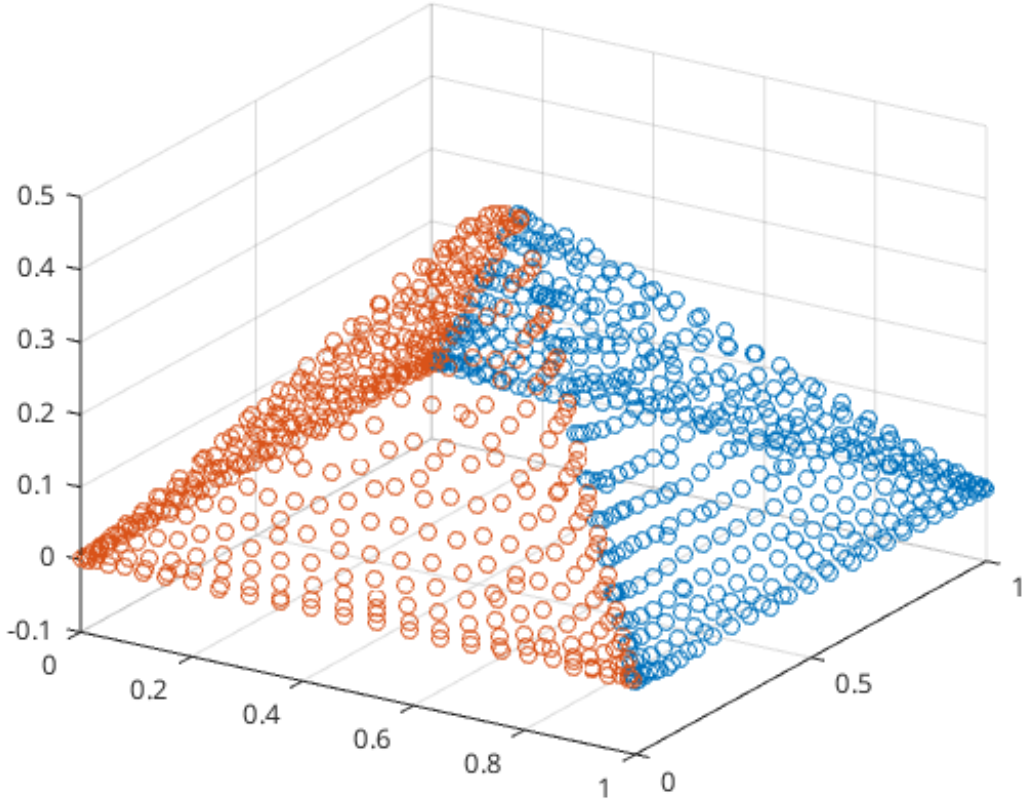


FIG. 5. Solution to (1.21) after 100 timesteps, with $\Delta\tau = 0.01$, $\xi = 0.01$ on the unit square $[0, 1]^2$ discretized with two triangles.

1.2.3. Pseudo-time relaxation with the ETD RK2 method. We start by describing ETD RK2 on a scalar semi-linear equation;

$$(1.33) \quad \frac{du}{d\tau} = Lu + Nu,$$

with $u(\tau^n) = u^n$, L a scalar, and N a nonlinear scalar valued function of u . The ETD RK2 temporal integrator is specifically designed to allow larger time steps for stiff problems with a non-linear part, and exhibits second-order accuracy in time. The update proceeds in two stages, delineated below, where the time step size is $d\tau$.

(i) Compute the exponential step

$$\phi_1 = e^{d\tau L} u^n$$

(ii) Evaluate the nonlinearity

$$N_1 = N(u^n)$$

(iii) Midpoint update

$$\Phi_2 = \phi_1 + d\tau \varphi(d\tau L) N_1$$

(iv) Evaluate nonlinearity at midpoint

$$N_2 = N(\Phi_2)$$

(v) Final update

$$u^{n+1} = \phi_1 + d\tau \varphi_1(d\tau L) N_1 + d\tau \varphi_2(d\tau L) (N_2 - N_1)$$

Above, φ_1, φ_2 are entire functions defined by

$$(1.34) \quad \varphi_1(z) = \frac{e^z - 1}{z}$$

$$(1.35) \quad \varphi_2(z) = \frac{e^z - 1 - z}{z^2}.$$

Now, when we move to the DAE system (1.30), several issues arise. To examine these issues, let's express the system in the form of (1.33);

$$(1.36) \quad \begin{aligned} \frac{d\mathbf{U}}{d\tau} &= L\mathbf{U} + \tilde{\mathbf{N}}(\mathbf{U}), \quad L = -\xi\mathcal{K}, \quad \tilde{\mathbf{N}}(\mathbf{U}) = \mathbf{F} - \mathbf{N}(\mathbf{U}) \\ B_{id}\mathbf{U} &= \mathbf{G}_{bc}. \end{aligned}$$

Above, we have simply absorbed the negative signs and load function. If L were invertible, then (1.34)-(1.35) have clear generalization for matrix-valued input, involving the inverse of L . However, we know that $\mathcal{K}$ is always rank degenerate in our formulation. The derivative of the constant polynomial is 0, so the first row and column of each non-zero block of $\mathcal{K}$ is 0. Hence, we must turn to the power-series definition of matrix valued functions. For φ_1, φ_2 , this is

$$\begin{aligned} \varphi_1(z) &= \sum_{k=0}^{\infty} \frac{z^k}{(k+1)!} \Rightarrow \varphi_1(d\tau L) = \sum_{k=0}^{\infty} \frac{(d\tau L)^k}{(k+1)!}, \\ \varphi_2(z) &= \sum_{k=0}^{\infty} \frac{z^k}{(k+2)!} \Rightarrow \varphi_2(d\tau L) = \sum_{k=0}^{\infty} \frac{(d\tau L)^k}{(k+2)!}. \end{aligned}$$

The convergence of these matrix-valued series is guaranteed if $\|d\tau L\| < 1$. For a fixed, ξ , this gives us a condition for selecting the timestep:

$$(1.37) \quad d\tau < \frac{1}{\xi\|\mathcal{K}\|}$$

In practice, we only compute the action of $\varphi_j(d\tau L)$ on vectors, as in steps (iii) and (iv) of the ETD RK2 method, and sum these mat-vecs term by term in the series. Another issue which arises concerns constraint enforcement at each step. The exponential matrix-valued functions will not, in general, preserve constraint enforcement. Hence, we must project the intermediate solutions at each stage onto the null-space³ of the constraint matrix B_{id} . To construct an orthogonal projector onto $\text{Null}(B_{id})$, we begin with a few definitions. An orthogonal projector $P \in \mathbb{R}^{n \times n}$ satisfies

$$(1.38) \quad (\text{idempotence}) \quad P^2 = P$$

$$(1.39) \quad (\text{symmetry}) \quad P^T = P.$$

³This assumes we have homogeneous Dirichlet conditions $\mathbf{G}_{bc} = 0$. When conditions are inhomogeneous, the projection remains essentially the same, but turns into an affine map, where we add to the projector the term $B_{id}^T (B_{id} B_{id}^T)^{-1} \mathbf{G}_{bc}$. This additional term is just the minimum-norm solution to $B_{id} \mathbf{U} = \mathbf{G}$. The overall form of the new projector is homogeneous projection + particular solution.

Suppose P_V is an orthogonal projector onto subspace V . Let $V^\perp = \{x \in \mathbb{R}^n | x \perp y \ \forall y \in V\}$ be the orthogonal complement of V . Then, $\forall x \in \mathbb{R}^n$, and any $y \in V$ observe that

$$\begin{aligned}\langle y, (I - P_V)x \rangle &= \langle y, x \rangle - \langle y, P_V x \rangle \\ &= \langle y, x \rangle - \langle P_V y, x \rangle \\ &= \langle y, x \rangle - \langle y, x \rangle \\ &= 0.\end{aligned}$$

Thus, $(I - P_V)x \in V^\perp$. Moreover,

$$\begin{aligned}(I - P_V)(I - P_V) &= I - 2P_V + P_V^2 = I - P_V, \\ (I - P_V)^T &= (I - P_V^T) = I - P_V,\end{aligned}$$

implies that $(I - P_V)$ is an orthogonal projector onto $V^\perp$. We also consider one of the results from the fundamental theorem of linear algebra. Suppose $x \in \text{Null}(B)$. Then $Bx = 0$. Consider $y \in \text{Row}(B)$. Then $y = B^T z$ for some $z \in \mathbb{R}^n$. We see that

$$\langle x, y \rangle = \langle x, B^T z \rangle = \langle Bx, z \rangle = 0,$$

which implies $x \in (\text{Row}(B))^\perp$. Now, let $x \in (\text{row}(B))^\perp$. For any $y \in \mathbb{R}^n$,

$$0 = \langle x, B^T y \rangle = \langle Bx, y \rangle.$$

If Bx is orthogonal to every $y \in \mathbb{R}^n$, then, $Bx = 0$, and $x \in \text{Null}(B)$. Overall, we have

$$\text{Null}(B) = (\text{Row}(B))^\perp.$$

So, for our case, we want an orthogonal projector P_V onto $V = \text{Null}(B_{\text{id}})$. We can construct such a projector by first building P_W , which projects orthogonally onto $W = \text{Row}(B_{\text{id}})$. Then, $P_V = I - P_W$, since $W^\perp = \text{Null}(B_{\text{id}})$. We derive the first projector, for completeness sake, by considering $x \in \mathbb{R}^{MN_T}$ arbitrary, and searching for $\hat{x} \in \text{Row}(B_{\text{id}})$ which minimizes

$$\|x - \hat{x}\|^2.$$

That is, letting $\hat{x} \in B_{\text{id}}^T y$ for some $y \in \mathbb{R}^{n_\lambda}$, we want to solve

$$\min_{y \in \mathbb{R}^{n_\lambda}} \|x - B_{\text{id}}^T y\|^2.$$

This is a standard quadratic minimization problem, and since B_{id} has full row-rank n_λ , the necessary condition for a minimizer reads

$$\frac{d}{dy} \|x - B_{\text{id}}^T y\|^2 = -2B_{\text{id}}(x - B_{\text{id}}^T y) = 0 \Rightarrow y = (B_{\text{id}} B_{\text{id}}^T)^{-1} B_{\text{id}} x$$

So, the projector of x orthogonally onto $\text{Row}(B_{\text{id}})$ is

$$P_W = B_{\text{id}}^T (B_{\text{id}} B_{\text{id}}^T)^{-1} B_{\text{id}},$$

and the projector of x orthogonally onto $\text{Null}(B_{\text{id}})$ is

$$(1.40) \quad P_V = I - B_{\text{id}}^T (B_{\text{id}} B_{\text{id}}^T)^{-1} B_{\text{id}}.$$

At this point, we have all the pieces in place to fully specify an ETRDK2 integrator for the DAE system (1.36).

(i) Compute the exponential step and projection

$$\begin{aligned}\phi_1 &= e^{d\tau L} \mathbf{U}^n \approx \sum_{k=0}^{N_{\text{tay}}} \frac{(d\tau L)^k \mathbf{U}^n}{k!} \\ \Phi_1 &= P_V \phi_1\end{aligned}$$

(ii) Evaluate the nonlinearity

$$\mathbf{N}_1 = \tilde{\mathbf{N}}(\mathbf{U}^n)$$

(iii) Midpoint update and projection

$$\begin{aligned}\phi_2 &= \Phi_1 + d\tau \varphi_1(d\tau L) \mathbf{N}_1 \approx \Phi_1 + d\tau \sum_{k=0}^{N_{\text{tay}}} \frac{(d\tau L)^k \mathbf{N}_1}{(k+1)!} \\ \Phi_2 &= P_V \phi_2\end{aligned}$$

(iv) Evaluate nonlinearity at midpoint

$$\mathbf{N}_2 = \tilde{\mathbf{N}}(\Phi_2)$$

(v) Final update and projection

$$\begin{aligned}\mathbf{U}^{n+1} &= \Phi_2 + P_V \left(d\tau \varphi_2(d\tau L)(\mathbf{N}_2 - \mathbf{N}_1) \right) \\ &\approx \Phi_2 + P_V \left(d\tau \sum_{k=0}^{N_{\text{tay}}} \frac{(d\tau L)^k (\mathbf{N}_2 - \mathbf{N}_1)}{(k+2)!} \right)\end{aligned}$$

At stages (i),(iii), and (v) of the method, we truncate the infinte-series expansion for the matrix exponential functions to N_{tay} terms, which is determined numerically by terminating the sum when the next term is within some tolerance.

2. Quadrature generation for Jacobi polynomials on the Simplex. Here, we detail the procedure for generating quadrature rules on the simplex. That is, given $f : \mathcal{T}^d \rightarrow \mathbb{R}$, we seek a set of N $(d+1)$ -tuples $(\mathbf{X}_1, \dots, \mathbf{X}_d, \mathbf{W})$ such that

$$(2.1) \quad \begin{aligned} & \int_{\mathcal{T}^d} f(x_1, \dots, x_d) w^{(\mathbf{a})}(x_1, \dots, x_d) dx_1 \cdots dx_d \\ & \approx \sum_{j=1}^N f(x_{j1}, \dots, x_{jd}) w_j, \end{aligned}$$

where $w^{(\mathbf{a})}(x_1, \dots, x_d)$ is the normalized Jacobi weight function on $\mathcal{T}^d$, and the approximation is exact for $M > N$ polynomials

Suppose that $\mathbf{P}(\mathbf{x}) \in \mathbb{R}^N$, with

$$N = \dim(\Pi_{n-1}^d) = \binom{n-1+d}{n-1},$$

as in (1.6) (in which we had $d = 2$), and $\mathbf{x} \in \mathbb{R}^d$. The orthonormal 3-term relation for Jacobi polynomials on the simplex is given by

$$(2.2) \quad x_i \mathbb{P}_n = A_{n,i} \mathbb{P}_{n+1} + B_{n,i} \mathbb{P}_n + A_{n-1,i}^T \mathbb{P}_{n-1}, \quad i = 1, \dots, d$$

$$(2.3) \quad A_{n,i} = \langle x_i \mathbb{P}_n, \mathbb{P}_{n+1}^T \rangle \in \mathbb{R}^{r_n^d \times r_{n+1}^d},$$

$$(2.4) \quad B_{n,i} = \langle x_i \mathbb{P}_n, \mathbb{P}_n^T \rangle \in \mathbb{R}^{r_n^d \times r_n^d},$$

where $\langle \cdot \rangle$ is the weighted L^2 inner product. The matrix recurrence generates an analog of the Jacobi matrix from 1D, except now there are d of them - one for each dimension:

$$(2.5) \quad \mathcal{J}_{n,i} = \begin{bmatrix} B_{0,i} & A_{0,i} & & & \\ A_{0,i}^T & B_{1,i} & A_{1,i} & & \\ & A_{1,i}^T & \ddots & \ddots & \\ & & \ddots & B_{n-2,i} & A_{n-2,i} \\ & & & A_{n-2,i}^T & B_{n-1,i} \end{bmatrix}, \quad i = 1, \dots, d$$

with $\mathcal{J}_{n,i} \in \mathbb{R}^{N \times N}$ symmetric. By definition, we see that the Jacobi matrices satisfy

$$(2.6) \quad x_i \mathbf{P} = \mathcal{J}_{n,i} \mathbf{P} + \begin{bmatrix} \mathbf{0}_{(N-r_{n-1}^d) \times 1} \\ A_{n-1,i} \mathbb{P}_n \end{bmatrix}, \quad i = 1, \dots, d.$$

We hope for a way to generalize the univariate Golub-Welsch algorithm for determining Gaussian quadrature from orthogonal polynomials on the line to the multi-variate case on the simplex. We say that $\Lambda = (\lambda_1, \dots, \lambda_d)$ is a *joint eigenvalue* of $J_{n,1}, \dots, J_{n,d}$ if there is a vector $\xi \neq 0$ such that $J_{n,i} \xi = \lambda_i \xi$ for $i = 1, \dots, d$. The vector ξ is the *joint eigenvector*. If there are N distinct joint eigenvalues, then the weights and abscissa for the Gaussian quadrature are as in 1D - the nodes are the joint eigenvalues, and the weights are the square of the first component of the corresponding joint eigenvector. However, Theorem 3.7.5 of Dunkl-Xu states that a necessary and sufficient condition for this to occur is the following commutativity relation

$$(2.7) \quad A_{n-1,i} A_{n-1,j}^T = A_{n-1,j} A_{n-1,i}^T, \quad 1 \leq i, j \leq d$$

This is equivalent to stating that $J_{n,i}$ are a commuting family, and hence, simultaneously diagonalizable. By inspection, we see that the coefficient matrices in the recurrence for Jacobi polynomials on the triangle do not, in general, satisfy the commutativity relation. Thus, *there do not exist minimal Gaussian quadrature rules on the Triangle using the classic orthogonal polynomials*.

We need an accurate quadrature on the triangle, as the convergence of any modal solver depends highly on accurate representations of the right-hand-side in the modal basis. While $J_{n,i}$ may not commute, there is a connection between their spectrums and non-minimal Gaussian-like quadrature rules on $\mathcal{T}^d$. Vioreanu and Rokhlin show that the joint eigenvalues of the combined complex operator

$$(2.8) \quad \mathcal{J}_n = \mathcal{J}_{n,1} + \iota \mathcal{J}_{n,2}$$

lie within $\mathcal{T}^2$ (Theorem 3.4 in VRQUAD). The algorithm described in their work uses the constant unit weight function and gives an alternate way to evaluate $\mathcal{J}_n$. For the case of the weighted space $L^2(\mathcal{T}, w_{a,b,c} W(x, y))$ with normalized weight (see (1.2)), the entries are

$$(2.9) \quad (\mathcal{J}_n)_{i,j} = \int_{\mathcal{T}} (x_1 + \iota x_2) P_j(x, y) P_i(x, y) w_{a,b,c} W(x, y) dx dy, \quad 0 \leq i, j \leq N-1,$$

where P_j is the j^{th} component of $\mathbf{P} = (\mathbb{P}_0^T, \mathbb{P}_1^T, \dots, \mathbb{P}_{n-1}^T)^T = (P_0^0, P_0^1, P_1^1, \dots, P_n^n)^T \in \mathbb{R}^N$. These are precisely the same operators defined in (2.8). However, this “complexification” trick is not available to us in dimensions $d > 2$, but there do exist algorithms for finding *approximate* joint diagonalizations of non-commuting matrix families.

2.1. Approximate joint diagonalization of non-commuting symmetric matrices. The approximate joint diagonalization problem for a set of real-valued symmetric matrices $\{C^1, \dots, C^d\}$ is to find the orthogonal matrix U such that the matrices

$$(2.10) \quad F^k = UC^kU^T, \quad 1 \leq k \leq d$$

are approximately diagonal. Alternatively, we can find the diagonal matrices F^k such that C^k are approximately equal to $U^T F^k U$. The definition of “as diagonal as possible” leads to various formulations of the diagonalization problem in terms of minimizing an objective function. For example, solving

$$(2.11) \quad \min_{\substack{U \in \mathbb{R}^N \\ U^T U = U U^T = I}} \sum_{i \neq j} (F_{ij}^k)^2$$

is equivalent to minimizing the off-diagonal entries of F^k by orthogonal similarity transformations of C^k . On the other hand, solving

$$(2.12) \quad \min_{\substack{U, F^1, \dots, F^d \\ U^T U = U U^T = I}} \sum_{k=1}^d \|C^k - U^T F^k U\|_F^2$$

is finding the best representation of C^k in the basis of U - a subspace fitting approach. For both problems, we require orthogonality of U , as we want it to represent the “average eigenstructure” of C^k . For the Jacobi matrices $J_{n,i}$, this requirement is natural. We want a decomposition close to the optimal quadrature case of exact joint diagonalization. For this purpose, we use the algorithm specified in “Jacobi Angles for Simultaneous Diagonalization”. The joint-eigenvalues provide us with an initial guess to a subsequent optimization problem.

2.2. Quadrature via roots. We want to integrate the M polynomials up to total degree $m - 1$ exactly with $m \geq n$ using only N nodes and weights. That is, with $M = \dim(\Pi_{m-1}^d) = \binom{m-1+d}{m-1}$, the integrals for which we want an exact quadrature are

$$(2.13) \quad \begin{aligned} \mathcal{I}_j &= \int_{\mathcal{T}^d} P_j(\mathbf{x}) w_{\mathbf{a}} W(\mathbf{x}) d(\mathbf{x}), \quad 0 \leq j \leq M - 1 \\ &= \langle P_j, P_0 \rangle = \delta_{j0}, \end{aligned}$$

where we use mutual orthonormality under $w_{a,b,c}W(x,y)$ (1.4) to evaluate them and arrive at the δ_{j0} term. If $\mathbf{X}_1, \dots, \mathbf{X}_d, \mathbf{W} \in \mathbb{R}^N$ are the nodes and weights, $\mathcal{I} \in \mathbb{R}^M$ the above integrals, and $\mathbf{P}(\mathbf{x}) \in \mathbb{R}^M$ (with $\mathbf{x} \in \mathbb{R}^d$) our $M > N$ polynomials, the objective function we aim to find a zero of is $\mathbf{F} : \mathbb{R}^{N \times (d+1)} \rightarrow \mathbb{R}^M$ given by

$$(2.14) \quad \mathbf{F}(\mathbf{X}_1, \dots, \mathbf{X}_d, \mathbf{W}) = \mathbf{P}(\tilde{\mathbf{z}})\mathbf{W} - \mathbf{e}_1, \quad \mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \in \mathbb{R}^M, \quad \mathbf{P}(\tilde{\mathbf{z}}) \in \mathbb{R}^{M \times N},$$

where

$$(2.15) \quad \tilde{\mathbf{z}} = [\mathbf{X}_1 \quad \dots \quad \mathbf{X}_d] \in \mathbb{R}^{N \times d}$$

Now, letting

$$(2.16) \quad \mathbf{z} = [\tilde{\mathbf{z}} \quad \mathbf{W}] \in \mathbb{R}^{N \times (d+1)},$$

we can use the Gauss-Newton iteration (where A^+ is the Moore-Penrose pseudoinverse)

$$(2.17) \quad \mathbf{z}^{k+1} = \mathbf{z}^k - \alpha_k [\nabla(\mathbf{F}(\mathbf{z}^k))]^+ \mathbf{F}(\mathbf{z}^k),$$

to find roots of (2.14), where $\alpha \in (0, 1)$. As we show in Section 2.2.3, we have access to analytical formula for computing the derivatives of (2.14), which enables us to alternatively use the damped Newton iteration

$$(2.18) \quad \mathbf{z}^{k+1} = \mathbf{z}^k - \alpha [\mathcal{D}^2(\mathbf{F}(\mathbf{z}^k))]^{-1} \nabla(\mathbf{F}(\mathbf{z}^k)).$$

This iteration does not work well, as the Hessian for each component of the objective is not well conditioned, i.e. near singular. Using the convex problem is a better approach. However, the Gauss-Newton iteration gives rapid convergence towards the solution, albeit with violation of the constraints. It might be wise to use Gauss-Newton to further initialize the guess for the solution, before passing it to iteration (2.20). We also consider the Newton iteration for the equivalent problem of finding roots of the function $f : \mathbb{R}^{N \times (d+1)} \rightarrow \mathbb{R}$ defined by

$$(2.19) \quad f(\mathbf{z}) := \frac{1}{2} \|\mathbf{F}(\mathbf{z})\|_2^2,$$

with iteration

$$(2.20) \quad \mathbf{z}^{k+1} = \mathbf{z}^k - \alpha [\mathcal{D}^2(f(\mathbf{z}^k))]^{-1} \nabla(f(\mathbf{z}^k))$$

The choice of initial point $\mathbf{z}^0 = (\tilde{\mathbf{z}}^0, \mathbf{W}_0)$ and control of the step length α_k are vital for its convergence. Moreover, we'd like to incorporate inequality constraints on $\mathbf{z}^k$ to enforce that quadrature nodes are interior to $\mathcal{T}^2$. Positivity of weights is also desirable, though not absolutely necessary for stability when the absolute value of each weight is not large. By casting the problem of finding roots of (2.14), (2.19) into a minimization problem, we can use techniques from constrained optimization to find acceptable quadratures.

First, we make a few observations about the objective function (2.19). Expanding out the norm in terms of the partitioning of the variable $\mathbf{z}$ defined in (2.16), we have

$$(2.21) \quad f(\mathbf{z}) = \frac{1}{2} \mathbf{W}^T \mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}}) \mathbf{W} - (\mathbf{1}_{N \times 1}^T \mathbf{W}) + \frac{1}{2}.$$

Since $M > N$, and each of the M polynomials is independent, $\mathbf{P}(\tilde{\mathbf{z}})$ has full column rank for any $\tilde{\mathbf{z}} \in \mathbb{R}^{N \times d}$ comprised of N distinct points in $\mathcal{T}^d \subset \mathbb{R}^d$. This implies that $\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})$ is positive definite, and (2.21) is a quadratic, strongly convex function in $\mathbf{W}$. The optimal $\mathbf{W}$ is easily determined given the necessary and sufficient optimality condition for strongly convex functions; namely, $\nabla_{\mathbf{W}} f(\mathbf{z}) = 0$. Hence,

$$(2.22) \quad \begin{aligned} \nabla_{\mathbf{W}} f(\mathbf{z}) &= \mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}}) \mathbf{W} - \mathbf{1}_{N \times 1} = 0 \\ \Rightarrow \mathbf{W} &= [\mathbf{P}(\tilde{\mathbf{z}})^T (\mathbf{P}(\tilde{\mathbf{z}}))]^{-1} \mathbf{1}_{N \times 1}. \end{aligned}$$

Inserting (2.22) into (2.14) gives

$$\begin{aligned} \tilde{\mathbf{F}}(\tilde{\mathbf{z}})^T &= \mathbf{1}_{1 \times N} [\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})]^{-1} \mathbf{P}(\tilde{\mathbf{z}})^T - \mathbf{e}_1^T \\ \Rightarrow \mathbf{F}(\tilde{\mathbf{z}}) &= \mathbf{P}(\tilde{\mathbf{z}})^\dagger \mathbf{1}_{N \times 1} - \mathbf{e}_1 \end{aligned}$$

The condition number of $\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})$ grows as the polynomial order increases. Hence, we will need some kind of preconditioning before computing the weights. In particular, the pre-conditioning can be a derivative-free optimization on the condition number of $\mathbf{P}^T(\tilde{\mathbf{z}})$ for a fixed $\mathbf{W}$, ensuring the value of $f(\mathbf{z})$ does not exceed its initial value. I've already implemented this with NLOPT, and it works well. .

2.2.1. Roots via minimization. The optimization problem we would like to solve, stated in canonical form, is

$$(2.23) \quad \begin{aligned} \min_{\mathbf{z} \in \mathbb{R}^{N \times (d+1)}} f(\mathbf{z}) \quad \text{such that} \\ f_i(\mathbf{z}) \leq 0, \quad i = 1, \dots, (d+1)N \end{aligned}$$

where $\mathbf{f} : \mathbb{R}^{N \times (d+1)} \rightarrow \mathbb{R}^{(d+1)N} := \begin{bmatrix} f_1(\mathbf{z}) \\ \vdots \\ f_{(d+1)N}(\mathbf{z}) \end{bmatrix}$ is given by

$$(2.24) \quad \mathbf{f}(\mathbf{z}) = G \cdot \text{vec}(\mathbf{z}) - \begin{bmatrix} \mathbf{0}_{dN \times 1} \\ \mathbf{1}_{N \times 1} \end{bmatrix}, \quad G = \begin{bmatrix} -I_1 & \times & \cdots & \times & \times \\ \times & \ddots & \ddots & \vdots & \vdots \\ \vdots & \ddots & \ddots & \vdots & \vdots \\ \times & \cdots & \cdots & -I_d & \times \\ I_1 & \cdots & \cdots & I_d & \times \end{bmatrix} \in \mathbb{R}^{(d+1)N \times (d+1)N},$$

and $\mathbf{f}$ is given by (2.19). The zero entries in the constraint matrix G are denoted by $\times$, and each $I_1, \dots, I_d$ is an $N \times N$ identity matrix. Specifically, each block row of the constraint equation $\mathbf{f}(\mathbf{z})$ corresponds, respectively, to the following inequality constraints for $j = 1 \dots N$:

- (i) $X_{1j} \geq 0, \dots, X_{dj} \geq 0$, which restricts the solution to the non-negative dN -orthant .
- (ii) $|\mathbf{X}_j|_1 \leq 1$, which is the L_1 -norm definition of the simplex $\mathcal{T}^d$, and means that no abscissa should be outside of it.

Now, we can optimize over specific partitions of the variable $\mathbf{z}$ independently, and formulate an equivalent problem. With $\tilde{\mathbf{z}}$ as in (2.15), define $\tilde{f} : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}$ by

$$(2.25) \quad \begin{aligned} \tilde{f}(\tilde{\mathbf{z}}) &= \inf \{ f(\begin{bmatrix} \tilde{\mathbf{z}} & \mathbf{W} \end{bmatrix}) \mid \mathbf{W} \in \mathbb{R}^N \} \\ &= f(\tilde{\mathbf{z}}, [\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})]^{-1} \mathbf{1}_{N \times 1}) \\ &= \frac{1}{2} (\mathbf{1}_{1 \times N} [\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})]^{-1} \mathbf{1}_{N \times 1} - 2(\mathbf{1}_{1 \times N} [\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})]^{-1} \mathbf{1}_{N \times 1}) + 1) \\ &= -\frac{1}{2} (\mathbf{1}_{1 \times N} [\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})]^{-1} \mathbf{1}_{N \times 1} - 1) \end{aligned}$$

where we inserted the optimal $\mathbf{W}$ found in (2.22). The problem (2.23) is then equivalent to

$$(2.26) \quad \begin{aligned} \min_{\tilde{\mathbf{z}} \in \mathbb{R}^{N \times d}} \tilde{f}(\tilde{\mathbf{z}}) \quad \text{such that} \\ \tilde{f}_i(\tilde{\mathbf{z}}) \leq 0, \quad i = 1, \dots, (d+1)N \end{aligned}$$

where $\tilde{\mathbf{f}} : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^{(d+1)N} := \begin{bmatrix} \tilde{f}_1(\tilde{\mathbf{z}}) \\ \vdots \\ \tilde{f}_{(d+1)N}(\tilde{\mathbf{z}}) \end{bmatrix}$ is given by

$$(2.27) \quad \tilde{\mathbf{f}}(\tilde{\mathbf{z}}) = G \cdot \text{vec}(\tilde{\mathbf{z}}) - \begin{bmatrix} \mathbf{0}_{dN \times 1} \\ \mathbf{1}_{N \times 1} \end{bmatrix}, \quad G = \begin{bmatrix} -I_1 & \times & \cdots & \times \\ \times & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ \times & \cdots & \cdots & -I_d \\ I_1 & \cdots & \cdots & I_d \end{bmatrix} \in \mathbb{R}^{(d+1)N \times dN},$$

Let $L : \mathbb{R}^{dN} \times \mathbb{R}^{(d+1)N} \rightarrow \mathbb{R}$ denote the *Lagrangian* of the *primal objective* (2.26). With *dual variables* $\boldsymbol{\lambda} \in \mathbb{R}^{(d+1)N}$, it is given by

$$(2.28) \quad L(\tilde{\mathbf{z}}, \boldsymbol{\lambda}) = \tilde{f}(\tilde{\mathbf{z}}) + \boldsymbol{\lambda}^T \tilde{\mathbf{f}}(\tilde{\mathbf{z}}).$$

The *Lagrange dual function* of (2.28) is defined as the minimum value of the Lagrangian over $\tilde{\mathbf{z}}$:

$$(2.29) \quad g(\boldsymbol{\lambda}) = \inf_{\tilde{\mathbf{z}} \in \mathbb{R}^{dN}} L(\tilde{\mathbf{z}}, \boldsymbol{\lambda}).$$

The dual function gives a lower bound on the optimal value p^* of the optimization problem (2.26). It stands to reason that there may exist a “best” lower bound, obtainable through solving the *Lagrange dual problem*:

$$(2.30) \quad \begin{aligned} & \max_{\boldsymbol{\lambda} \in \mathbb{R}^{(d+1)N}} g(\boldsymbol{\lambda}) \\ & \text{subject to } \boldsymbol{\lambda} \succeq 0 \end{aligned}$$

The dual problem is a concave optimization problem, since the objective $g(\boldsymbol{\lambda})$ is the pointwise infimum of a family of affine functions of $\boldsymbol{\lambda}$ (they are the sums of hyperplanes with normal $\boldsymbol{\lambda}$). This holds regardless of the convexity of the original objective $\tilde{f}(\tilde{\mathbf{z}})$. Thus, there exists a notion of the *duality gap*. If p^* is the optimal value of the primal problem (2.26), and d^* is that of the dual problem (2.30), then the lower bound property of the dual problem implies, regardless of the convexity of the primal objective, that

$$(2.31) \quad d^* \leq p^*.$$

The difference $p^* - d^*$ is referred to as the duality gap. If the duality gap is zero, then we say that *strong duality* holds. That is, the bound obtained from the Lagrange dual function is tight, and solving the dual problem gives us a solution for the primal problem. The dual problem can be solved using the regular techniques of constrained convex optimization, though we have described it only for theoretical clarification.

If $\tilde{\mathbf{z}}^*$ and $(\boldsymbol{\lambda}^*)$ are primal and dual optimal points, respectively, with zero duality gap, then $\tilde{\mathbf{z}}^*$ minimizes $L(\tilde{\mathbf{z}}, \boldsymbol{\lambda}^*)$ over $\tilde{\mathbf{z}}$. Hence, the first order necessary optimality conditions must hold; namely, that the gradient of L must vanish:

$$(2.32) \quad \nabla \tilde{f}(\tilde{\mathbf{z}}^*) + \boldsymbol{\lambda}^{*T} \nabla \tilde{\mathbf{f}}(\tilde{\mathbf{z}}^*) = 0.$$

This leads to the following conditions characterizing the solution to any optimization problem for which strong duality holds, known as the Karush-Kuhn-Tucker (KKT) conditions, which we specify for (2.23):

$$(2.33) \quad \begin{aligned} \tilde{f}(\tilde{\mathbf{z}}^*) &\leq 0, \\ \boldsymbol{\lambda}^* &\succeq 0, \\ \boldsymbol{\lambda}^{*T} \tilde{\mathbf{f}}(\tilde{\mathbf{z}}^*) &= 0, \\ \nabla \tilde{f}(\tilde{\mathbf{z}}^*) + \boldsymbol{\lambda}^{*T} \nabla \tilde{\mathbf{f}}(\tilde{\mathbf{z}}^*) &= 0. \end{aligned}$$

If an optimal primal dual pair is found, $\mathbf{z}^*$ is not necessarily a zero of $\mathbf{F}(\mathbf{z})$. We have only satisfied the necessary KKT conditions. However, we may be closer to an actual root and will have maintained interior nodes and positive weights. We can use this new estimate $\mathbf{z}^*$ as an initial point in a Gauss-Newton iteration that may relax $\mathbf{F}(\mathbf{z})$ to zero quadratically.

We will recast (2.23) into a fully unconstrained problem using the barrier method, as described in the next section.

2.2.2. Barrier method. In the barrier method, we start by rewriting (2.26) in a form which incorporates the inequality constraints directly into the objective, as in

$$(2.34) \quad \min_{\tilde{\mathbf{z}} \in \mathbb{R}^{dN}} \tilde{f}(\tilde{\mathbf{z}}) + \sum_{i=1}^{(d+1)N} I_{-}(\tilde{f}_i(\tilde{\mathbf{z}}))$$

where $I_{-} : \mathbb{R} \rightarrow \mathbb{R}$ is the indicator function for the non-positive reals;

$$(2.35) \quad I_{-}(x) = \begin{cases} 0, & x \leq 0 \\ \infty, & x > 0 \end{cases}.$$

While (2.34) has no constraints, the objective is not differentiable. As such, we must approximate the indicator function with a sequence of differentiable functions converging to it in the pointwise limit:

$$(2.36) \quad \tilde{I}_-(x) = -(1/t) \log(-x), \quad t > 0, \quad x \in \mathbb{R}_*^-.$$

Note, we use the convention that $\tilde{I}_-(x > 0) = \infty$. Observe that $\tilde{I}_-(x) \rightarrow \infty$ as $x \rightarrow 0^-$. As $t \rightarrow \infty$ with $x < 0$, $\tilde{I}_-(x) \rightarrow 0$. The problem is now approximated with a differentiable objective:

$$(2.37) \quad \min_{\tilde{\mathbf{z}} \in \mathbb{R}^{dN}} t\tilde{f}(\tilde{\mathbf{z}}) + \phi(\tilde{\mathbf{z}})$$

for $t > 0$, where the log barrier function ϕ is given by

$$(2.38) \quad \phi(\tilde{\mathbf{z}}) = - \sum_{i=1}^{(d+1)N} \log(-\tilde{f}_i(\tilde{\mathbf{z}})).$$

We can solve (2.37) for increasing values of t , and use the solution from each previous value of t as the initialization for the problem with the next t value. This generates a sequence $\tilde{\mathbf{z}}^*(t)$, known as the *central path*, which converges to the optimal point $\tilde{\mathbf{z}}^*$ as $t \rightarrow \infty$ (if the problem is solvable). To each approximate problem for a given t , we can associate the Newton system to solve for the decrement $\Delta\tilde{\mathbf{z}}$:

$$(2.39) \quad (t\nabla^2\tilde{f}(\tilde{\mathbf{z}}) + \nabla^2\phi(\tilde{\mathbf{z}}))\Delta\tilde{\mathbf{z}} = -(t\nabla\tilde{f}(\tilde{\mathbf{z}}) + \nabla\phi(\tilde{\mathbf{z}}))$$

We provide the relevant derivatives here. First, we define $U : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^{N \times N}$ and $g : \mathbb{R}^{N \times N} \rightarrow \mathbb{R}$ by

$$g(U) = -\mathbf{1}_{N \times 1}^T U^{-1} \mathbf{1}_{N \times 1}, \quad U = \mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}}), \quad \tilde{\mathbf{z}} = \begin{bmatrix} z_{11} & \cdots & z_{1d} \\ \vdots & \ddots & \vdots \\ z_{N1} & \cdots & z_{Nd} \end{bmatrix}$$

We have the chain rule identity

$$(2.40) \quad \frac{\partial g(U)}{\partial z_{ij}} = \text{Tr} \left[\left(\frac{\partial g(U)}{\partial U} \right)^T \frac{\partial U}{\partial z_{ij}} \right].$$

We also know that

$$(2.41) \quad \frac{\partial g(U)}{\partial U} = U^{-T} \mathbf{1}_{N \times 1} \mathbf{1}_{N \times 1}^T U^{-T}.$$

Altogether, we have

$$\begin{aligned} U_{kl} &= \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) P_q(z_{l1}, \dots, z_{ld}) \\ \frac{\partial U_{kl}}{\partial z_{ij}} &= \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{ij}} P_q(z_{l1}, \dots, z_{ld}) + P_q(z_{l1}, \dots, z_{ld}) \partial_{z_{ij}} P_q(z_{k1}, \dots, z_{kd}) \\ &= \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{ij}} P_q(z_{l1}, \dots, z_{ld}) \delta_{il} + P_q(z_{l1}, \dots, z_{ld}) \partial_{z_{ij}} P_q(z_{k1}, \dots, z_{kd}) \delta_{ik} \\ \Rightarrow \frac{\partial U_{kl}}{\partial z_{ij}} &= \begin{cases} 0, & (i \neq l) \wedge (i \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{lj}} P_q(z_{l1}, \dots, z_{ld}), & (i = l) \wedge (i \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{l1}, \dots, z_{ld}) \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}), & (i \neq l) \wedge (i = k) \\ 2 \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}), & (i = l) \wedge (i = k) \end{cases} \\ &= \begin{cases} 0, & (i \neq l) \wedge (i \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \mathbf{P}^j(z_{l1}, \dots, z_{ld})^T \mathcal{D}_j \mathbf{e}_q, & (i = l) \wedge (i \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{l1}, \dots, z_{ld}) \mathbf{P}^j(z_{k1}, \dots, z_{kd})^T \mathcal{D}_j \mathbf{e}_q, & (i \neq l) \wedge (i = k) \\ 2 \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \mathbf{P}^j(z_{k1}, \dots, z_{kd})^T \mathcal{D}_j \mathbf{e}_q, & (i = l) \wedge (i = k) \end{cases} \\ &= \begin{cases} 0, & (i \neq l) \wedge (i \neq k) \\ \mathbf{P}(z_{k1}, \dots, z_{kd})^T \left(\mathcal{D}_j^T \mathbf{P}^j(z_{l1}, \dots, z_{ld}) \right), & (i = l) \wedge (i \neq k) \\ \mathbf{P}(z_{l1}, \dots, z_{ld})^T \left(\mathcal{D}_j^T \mathbf{P}^j(z_{k1}, \dots, z_{kd}) \right), & (i \neq l) \wedge (i = k) \\ 2 \mathbf{P}(z_{k1}, \dots, z_{kd})^T \left(\mathcal{D}_j^T \mathbf{P}^j(z_{k1}, \dots, z_{kd}) \right), & (i = l) \wedge (i = k) \end{cases} \end{aligned}$$

For the second derivatives, define $V : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^{N \times N}$ and $h : \mathbb{R}^{N \times N} \rightarrow \mathbb{R}^{N \times N}$ by

$$h(V) = V \mathbf{1}_{N \times 1} \mathbf{1}_{N \times 1}^T V, \quad V = U^{-1} = [\mathbf{P}(\tilde{\mathbf{z}})^T \mathbf{P}(\tilde{\mathbf{z}})]^{-1}$$

The differential trace identity gives

$$\partial \text{Tr}[h(V) \partial U] = \text{Tr}[\partial h(V) \partial U + h(V) \partial^2 U]$$

So,

$$\frac{\partial \text{Tr}(h(V) \frac{\partial U}{\partial z_{ij}})}{\partial z_{st}} = \text{Tr} \left[\frac{\partial h(V)}{\partial z_{st}} \frac{\partial U}{\partial z_{ij}} + h(V) \frac{\partial^2 U}{\partial z_{st} \partial z_{ij}} \right]$$

Now, as before for g , we find for h through the chain rule identity that

$$(2.42) \quad \frac{\partial h(V)_{kl}}{\partial z_{st}} = \text{Tr} \left[\left(\frac{\partial h(V)_{kl}}{\partial V} \right)^T \frac{\partial V}{\partial z_{st}} \right]$$

Since $V = V^T$, we know that

$$(2.43) \quad \begin{aligned} \frac{\partial h(V)}{\partial V} &:= \frac{\partial (V^T \mathbf{1}_{N \times 1} \mathbf{1}_{N \times 1}^T V)_{kl}}{\partial V_{ij}} \\ &= \delta_{lj} (V \mathbf{1}_{N \times 1} \mathbf{1}_{N \times 1}^T)_{ki} + \delta_{kj} (\mathbf{1}_{N \times 1} \mathbf{1}_{N \times 1}^T V)_{il}. \end{aligned}$$

We can also compute⁴

$$(2.44) \quad \frac{\partial V}{\partial z_{st}} = \frac{\partial (U^{-1})}{\partial z_{st}} = -U^{-1} \frac{\partial U}{\partial z_{st}} U^{-1}$$

⁴This can be seen by taking the derivative of $I = U(\tilde{\mathbf{z}})U(\tilde{\mathbf{z}})^{-1}$, which is 0, and solving for the desired term.

We handle the Hessian of U below:

$$\begin{aligned}
\frac{\partial^2 U_{kl}}{\partial z_{st} \partial z_{ij}} &= \begin{cases} 0, & (i \neq l) \wedge (i \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{st}} \partial_{z_{lj}} P_q(z_{l1}, \dots, z_{ld}) + \partial_{z_{st}} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{lj}} P_q(z_{l1}, \dots, z_{ld}), & (i = l) \wedge (i \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{l1}, \dots, z_{ld}) \partial_{z_{st}} \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}) + \partial_{z_{st}} P_q(z_{l1}, \dots, z_{ld}) \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}), & (i \neq l) \wedge (i = k) \\ 2 \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{st}} \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}) + \partial_{z_{st}} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}), & (i = l) \wedge (i = k) \end{cases} \\
&= \begin{cases} 0, & (i \neq l) \wedge (i \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{st}} \partial_{z_{lj}} P_q(z_{l1}, \dots, z_{ld}) \delta_{sl} + \partial_{z_{st}} P_q(z_{k1}, \dots, z_{kd}) \delta_{sk} \partial_{z_{lj}} P_q(z_{l1}, \dots, z_{ld}), & (i = l) \wedge (i \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{l1}, \dots, z_{ld}) \partial_{z_{st}} \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}) \delta_{sk} + \partial_{z_{st}} P_q(z_{l1}, \dots, z_{ld}) \delta_{sl} \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}), & (i \neq l) \wedge (i = k) \\ 2 \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{st}} \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}) \delta_{sk} + \partial_{z_{st}} P_q(z_{k1}, \dots, z_{kd}) \delta_{sk} \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}), & (i = l) \wedge (i = k) \end{cases} \\
&= \begin{cases} 0, & (i \neq l) \wedge (i \neq k) \\ 0, & (s \neq l) \wedge (s \neq k) \\ 0, & ((i = l) \wedge (i = k)) \wedge (s \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{lt}} \partial_{z_{lj}} P_q(z_{l1}, \dots, z_{ld}), & ((i = l) \wedge (i \neq k)) \wedge ((s = l) \wedge (s \neq k)) \\ \sum_{q=0}^{M-1} \partial_{z_{kt}} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{lj}} P_q(z_{l1}, \dots, z_{ld}), & ((i = l) \wedge (i \neq k)) \wedge ((s \neq l) \wedge (s = k)) \\ \sum_{q=0}^{M-1} \partial_{z_{lt}} P_q(z_{l1}, \dots, z_{ld}) \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}), & ((i \neq l) \wedge (i = k)) \wedge ((s = l) \wedge (s \neq k)) \\ \sum_{q=0}^{M-1} P_q(z_{l1}, \dots, z_{ld}) \partial_{z_{kt}} \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}), & ((i \neq l) \wedge (i = k)) \wedge ((s \neq l) \wedge (s = k)) \\ 2 \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{kt}} \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}) + & \\ 2 \sum_{q=0}^{M-1} \partial_{z_{kt}} P_q(z_{k1}, \dots, z_{kd}) \partial_{z_{kj}} P_q(z_{k1}, \dots, z_{kd}), & ((i = l) \wedge (i = k)) \wedge (s = k) \end{cases} \\
&= \begin{cases} 0, & (i \neq l) \wedge (i \neq k) \\ 0, & (s \neq l) \wedge (s \neq k) \\ 0, & ((i = l) \wedge (i = k)) \wedge (s \neq k) \\ \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \mathbf{P}^{jt}(z_{l1}, \dots, z_{ld})^T \mathcal{D}_{jt} \mathbf{e}_q, & ((i = l) \wedge (i \neq k)) \wedge ((s = l) \wedge (s \neq k)) \\ \sum_{q=0}^{M-1} (\mathbf{P}^t(z_{k1}, \dots, z_{kd})^T \mathcal{D}_t \mathbf{e}_q) (\mathbf{P}^j(z_{l1}, \dots, z_{ld}) \mathcal{D}_j \mathbf{e}_q), & ((i = l) \wedge (i \neq k)) \wedge ((s \neq l) \wedge (s = k)) \\ \sum_{q=0}^{M-1} (\mathbf{P}^t(z_{l1}, \dots, z_{ld})^T \mathcal{D}_t \mathbf{e}_q) (\mathbf{P}^j(z_{k1}, \dots, z_{kd}) \mathcal{D}_j \mathbf{e}_q), & ((i \neq l) \wedge (i = k)) \wedge ((s = l) \wedge (s \neq k)) \\ \sum_{q=0}^{M-1} P_q(z_{l1}, \dots, z_{ld}) \mathbf{P}^{jt}(z_{k1}, \dots, z_{kd})^T \mathcal{D}_{jt} \mathbf{e}_q, & ((i \neq l) \wedge (i = k)) \wedge ((s \neq l) \wedge (s = k)) \\ 2 \sum_{q=0}^{M-1} P_q(z_{k1}, \dots, z_{kd}) \mathbf{P}^{jt}(z_{k1}, \dots, z_{kd})^T \mathcal{D}_{jt} \mathbf{e}_q + & \\ 2 \sum_{q=0}^{M-1} (\mathbf{P}^t(z_{k1}, \dots, z_{kd}) \mathcal{D}_t \mathbf{e}_q) (\mathbf{P}^j(z_{k1}, \dots, z_{kd})^T \mathcal{D}_j \mathbf{e}_q), & ((i = l) \wedge (i = k)) \wedge (s = k) \end{cases} \\
&= \begin{cases} 0, & (i \neq l) \wedge (i \neq k) \\ 0, & (s \neq l) \wedge (s \neq k) \\ 0, & ((i = l) \wedge (i = k)) \wedge (s \neq k) \\ \mathbf{P}(z_{k1}, \dots, z_{kd})^T (\mathcal{D}_{jt}^T \mathbf{P}^{jt}(z_{l1}, \dots, z_{ld})), & ((i = l) \wedge (i \neq k)) \wedge ((s = l) \wedge (s \neq k)) \\ (\mathcal{D}_t^T \mathbf{P}^t(z_{k1}, \dots, z_{kd}))^T (\mathcal{D}_j^T \mathbf{P}^j(z_{l1}, \dots, z_{ld})), & ((i = l) \wedge (i \neq k)) \wedge ((s \neq l) \wedge (s = k)) \\ (\mathcal{D}_t^T \mathbf{P}^t(z_{l1}, \dots, z_{ld}))^T (\mathcal{D}_j^T \mathbf{P}^j(z_{k1}, \dots, z_{kd})), & ((i \neq l) \wedge (i = k)) \wedge ((s = l) \wedge (s \neq k)) \\ \mathbf{P}(z_{l1}, \dots, z_{ld})^T (\mathcal{D}_{jt}^T \mathbf{P}^{jt}(z_{k1}, \dots, z_{kd})), & ((i \neq l) \wedge (i = k)) \wedge ((s \neq l) \wedge (s = k)) \\ 2 \mathbf{P}(z_{k1}, \dots, z_{kd})^T (\mathcal{D}_{jt}^T \mathbf{P}^{jt}(z_{k1}, \dots, z_{kd})) + & \\ 2 (\mathcal{D}_t^T \mathbf{P}^t(z_{k1}, \dots, z_{kd}))^T (\mathcal{D}_j^T \mathbf{P}^j(z_{k1}, \dots, z_{kd})), & ((i = l) \wedge (i = k)) \wedge (s = k) \end{cases}
\end{aligned}$$

We verify the correctness of our formulae for the derivatives of our objective by comparing with second-order finite difference approximations. In particular, we show in Figure 6 the relative error in the 2-norm between the analytical evaluation and the finite difference approximation. Since the difference scheme is second order, we expect the error to behave like $\mathcal{O}(h^2)$, where h is the stencil width. That is, we should observe that $\log(\|\mathcal{D}(\mathbf{F}) - \hat{\mathcal{D}}(\mathbf{F})\|) \approx 2 \log(h) + \beta$, where β is the order constant, and $\hat{\mathcal{D}}(\mathbf{F})$ is the finite difference approximation. On a log – log scale, this means the slope of the error line (as a function of stencil width h) should be around 2. Indeed, we do observe this relationship in the regime of asymptotic convergence, before round-off error dominates truncation error for the finite difference scheme.

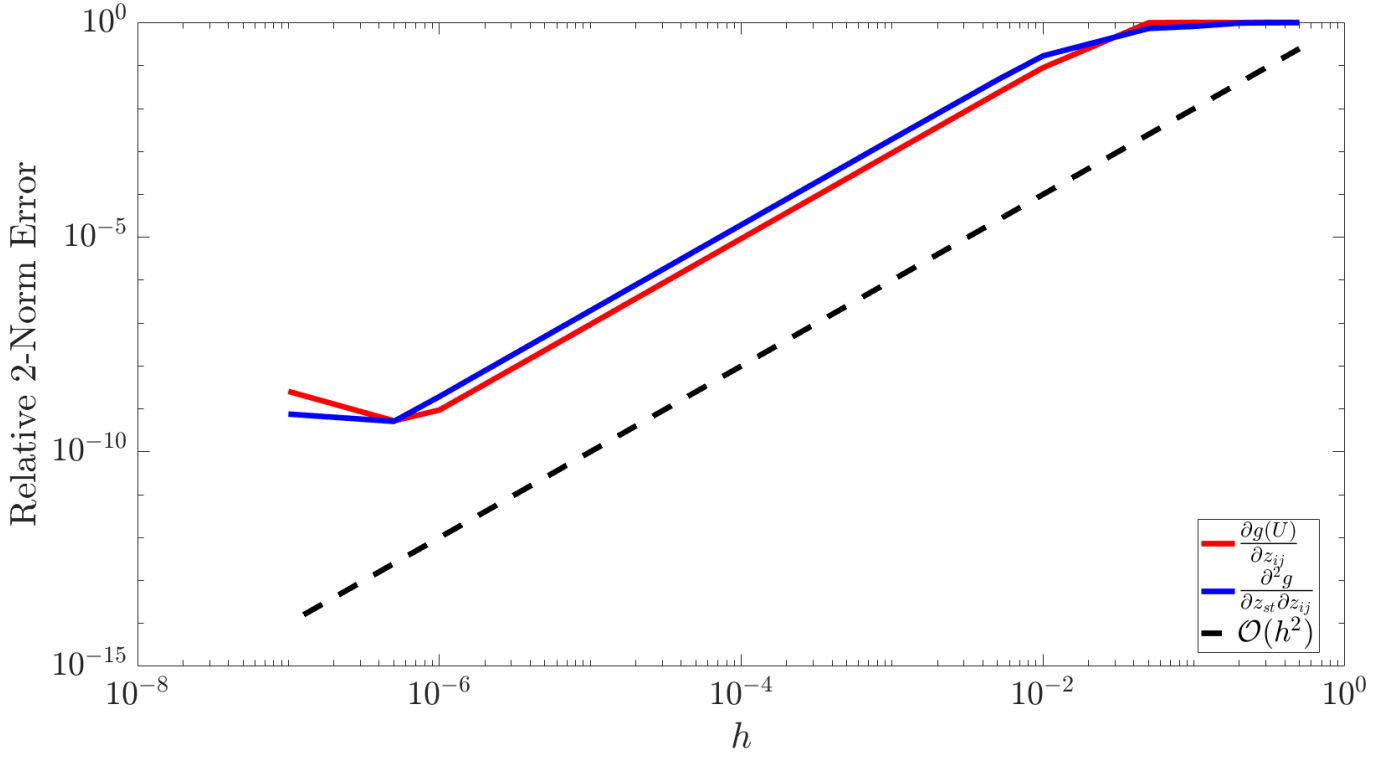


FIG. 6. Relative error in the 2-norm (Frobenius for matrix valued derivatives) of the analytical derivative formulae when compared to second order finite difference approximations. We see that our formulae must be correct, if our finite difference implementation is correct (which is an initial assumption). The kink in the error curves after which the error increases is typical of a finite difference scheme, and occurs at the “usual” location for at least twice differentiable functions, i.e. $h \approx 1e-6$. This corresponds to the point where numerical round-off error (due to IEEE floating point specification) dominates truncation error.

For this sort of simple “reverse” verification test, we assume that we don’t know if our analytical formulae are correct, and that a *verified* finite difference implementation is available. Then, we are essentially re-verifying the finite difference implementation, which on success, certifies correctness of the analytical formulae. The derivatives of the log-barrier function (2.38) are given by

$$\begin{aligned}
 \nabla \phi(\tilde{\mathbf{z}}) &= \sum_{i=1}^{(d+1)N} \frac{-1}{\tilde{f}_i(\tilde{\mathbf{z}})} \nabla \tilde{f}_i(\tilde{\mathbf{z}}) \\
 &= -G^T (\mathbf{1}_{(d+1)N \times 1} \odot \tilde{\mathbf{f}}(\tilde{\mathbf{z}})) \\
 \nabla^2 \phi(\tilde{\mathbf{z}}) &= \sum_{i=1}^{(d+1)N} \frac{1}{\tilde{f}_i^2(\tilde{\mathbf{z}})} \nabla \tilde{f}_i(\tilde{\mathbf{z}}) \nabla \tilde{f}_i(\tilde{\mathbf{z}})^T - \sum_{i=1}^{(d+1)N} \frac{1}{\tilde{f}_i(\tilde{\mathbf{z}})} \nabla^2 \tilde{f}_i(\tilde{\mathbf{z}}) \\
 &= \sum_{i=1}^{(d+1)N} \frac{1}{\tilde{f}_i^2(\tilde{\mathbf{z}})} \nabla \tilde{f}_i(\tilde{\mathbf{z}}) \nabla \tilde{f}_i(\tilde{\mathbf{z}})^T \\
 &= G^T [(\mathbf{1}_{(d+1)N \times 1} \odot \tilde{\mathbf{f}}(\tilde{\mathbf{z}}))^{\circ 2} \odot G]
 \end{aligned}
 \tag{2.45}$$

where we eliminated the second summation in the formula for the hessian of ϕ using the fact that each $\tilde{f}_i$ is linear in $\tilde{\mathbf{z}}$. These are verified in Figure 7.

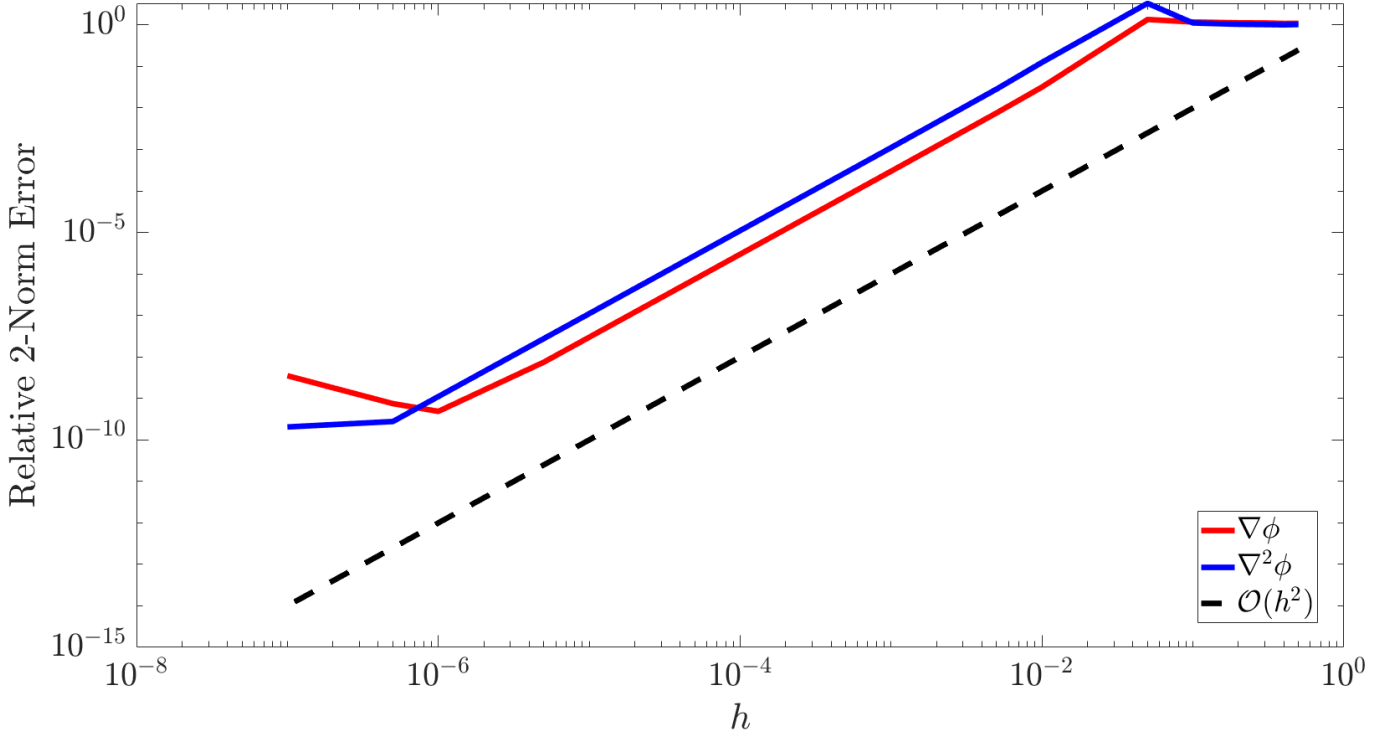


FIG. 7. Relative error in the 2-norm (Frobenius for matrix valued derivatives) of the analytical derivative formulae for the log-barrier function when compared to second order finite difference approximations.

2.2.3. Derivatives of the objective functions. We can use the machinery developed in Section 1.1 to compute derivatives of (2.14). Given that these tools are only at our disposal for $d \leq 2$, we proceed for now with $d = 2$ and $\mathbf{x} \in \mathcal{T}^2$. We view each component of the objective as a function $F_j : \mathbb{R}^{3N} \rightarrow \mathbb{R}$, and note that $\mathbf{P}$ is defined in the (a, b, c) Jacobi basis. Applying the derivative operators defined in (1.8), we have at a point $(\mathbf{X}, \mathbf{Y}, \mathbf{W})$,

$$\begin{aligned}
 F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W}) &= \mathbf{e}_j^T \mathbf{P}(\mathbf{X}, \mathbf{Y}) \mathbf{W} - \delta_{1j} = \sum_{i=1}^N P_j(x_i, y_i) w_i - \delta_{1j}, \\
 \frac{\partial F_j}{\partial x_k} &= w_k \mathbf{P}^x(x_k, y_k)^T \mathcal{D}_x \mathbf{e}_j, \quad \frac{\partial F_j}{\partial y_k} = w_k \mathbf{P}^y(x_k, y_k)^T \mathcal{D}_y \mathbf{e}_j, \quad \frac{\partial F_j}{\partial w_k} = \mathbf{P}(x_k, y_k)^T \mathbf{e}_j, \\
 \frac{\partial^2 F_j}{\partial x_l \partial x_k} &= w_k \mathbf{P}^{xx}(x_k, y_k)^T \mathcal{D}_{xx} \mathbf{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial x_l \partial y_k} = w_k \mathbf{P}^{xy}(x_k, y_k)^T \mathcal{D}_{xy} \mathbf{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial x_l \partial w_k} = \mathbf{P}^x(x_k, y_k)^T \mathcal{D}_x \mathbf{e}_j \delta_{lk}, \\
 \frac{\partial^2 F_j}{\partial y_l \partial x_k} &= w_k \mathbf{P}^{yx}(x_k, y_k)^T \mathcal{D}_{yx} \mathbf{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial y_l \partial y_k} = w_k \mathbf{P}^{yy}(x_k, y_k)^T \mathcal{D}_{yy} \mathbf{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial y_l \partial w_k} = \mathbf{P}^y(x_k, y_k)^T \mathcal{D}_y \mathbf{e}_j \delta_{lk}, \\
 \frac{\partial^2 F_j}{\partial w_l \partial x_k} &= \mathbf{P}^x(x_k, y_k)^T \mathcal{D}_x \mathbf{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial w_l \partial y_k} = \mathbf{P}^y(x_k, y_k)^T \mathcal{D}_y \mathbf{e}_j \delta_{lk}, \quad \frac{\partial^2 F_j}{\partial w_l \partial w_k} = 0 \cdot \delta_{lk},
 \end{aligned}$$

where

$$\begin{aligned}
 \mathcal{D}_x &= \mathcal{D}_{x, (a, b, c)}^{a+1, b, c+1}, \quad \mathbf{P}^x = \mathbf{P}^{(a+1, b, c+1)}, \quad \mathcal{D}_y = \mathcal{D}_{y, (a, b, c)}^{a, b+1, c+1}, \quad \mathbf{P}^y = \mathbf{P}^{(a, b+1, c+1)} \\
 \mathcal{D}_{xx} &= \mathcal{D}_{x, (a+1, b, c+1)}^{(a+2, b, c+2)} \mathcal{D}_{x, (a, b, c)}^{(a+1, b, c+1)}, \quad \mathbf{P}^{xx} = \mathbf{P}^{(a+2, b, c+2)}, \\
 \mathcal{D}_{yy} &= \mathcal{D}_{y, (a, b+1, c+1)}^{(a, b+2, c+2)} \mathcal{D}_{y, (a, b, c)}^{(a, b+1, c+1)}, \quad \mathbf{P}^{yy} = \mathbf{P}^{(a, b+2, c+2)} \\
 \mathcal{D}_{xy} &= \mathcal{D}_{x, (a, b+1, c+1)}^{(a+1, b+1, c+2)} \mathcal{D}_{y, (a, b, c)}^{(a, b+1, c+1)}, \quad \mathbf{P}^{xy} = \mathbf{P}^{(a+1, b+1, c+2)}, \\
 \mathcal{D}_{yx} &= \mathcal{D}_{y, (a+1, b, c+1)}^{(a+1, b+1, c+2)} \mathcal{D}_{x, (a, b, c)}^{(a+1, b, c+1)}, \quad \mathbf{P}^{yx} = \mathbf{P}^{xy}
 \end{aligned}$$

Now, reverting to the view of the objective as $\mathbf{F} : \mathbb{R}^{3N} \rightarrow \mathbb{R}^M$, we can write

$$(2.46) \quad [\mathcal{D}(\mathbf{F})]_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} = [\mathcal{D}_x^T \mathbf{P}^x(\mathbf{X}, \mathbf{Y}) \odot \mathbf{W} \quad \mathcal{D}_y^T \mathbf{P}^y(\mathbf{X}, \mathbf{Y}) \odot \mathbf{W} \quad \mathbf{P}(\mathbf{X}, \mathbf{Y})] \in \mathbb{R}^{M \times 3N},$$

and we can use this formula in the Gauss-Newton update (2.17). If we instead consider the equivalent problem of finding zeros of the objective norm $f : \mathbb{R}^{3N} \rightarrow \mathbb{R}$, $\mathbf{f}(\mathbf{X}, \mathbf{Y}, \mathbf{W}) = \|\mathbf{F}\|_2^2$, (solved by the iteration (2.20)), we must chain out derivatives to find the mapping $\mathcal{D}(\mathbf{f}) : \mathbb{R}^{3N} \rightarrow L(\mathbb{R}^{3N}, \mathbb{R})$ at a point $(\mathbf{X}, \mathbf{Y}, \mathbf{W})$ is given by

$$(2.47) \quad [\mathcal{D}(f)]_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} = 2\mathbf{F}(\mathbf{X}, \mathbf{Y}, \mathbf{W})^T [\mathcal{D}(\mathbf{F})]_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} \in \mathbb{R}^{1 \times 3N},$$

where $L(U, V)$ is the space of linear mappings between finite dimensional vector spaces U, V . In this case, the Hessian matrix of $\mathbf{f}$ is a mapping $\mathcal{D}^2(\mathbf{f}) : \mathbb{R}^{3N} \rightarrow L(\mathbb{R}^{3N}, \mathbb{R}^{3N})$ given at a point $(\mathbf{X}, \mathbf{Y}, \mathbf{W})$ by

$$(2.48) \quad \begin{aligned} [\mathcal{D}^2(f)]|_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} &= 2 \sum_{j=1}^M F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W}) [\mathcal{D}^2(F_j)]|_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} \\ &+ 2 \left([\mathcal{D}(\mathbf{F})]|_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} \right)^T [\mathcal{D}(\mathbf{F})]|_{(\mathbf{X}, \mathbf{Y}, \mathbf{W})} \\ &\in \mathbb{R}^{3N \times 3N} \end{aligned}$$

The vector Hessian of $\mathbf{F}$ appears in (2.48), and we can make sense of its tensor structure by reviewing the mixed partial formulae given at the beginning of this section. In particular, we have

$$(2.49) \quad \partial_{\mathbf{X}\mathbf{X}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left(\mathbf{1}_N^T \otimes \left(\mathbf{W} \odot \mathbf{P}^{xx}(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_{xx} \mathbf{e}_j \right) \right),$$

$$(2.50) \quad \partial_{\mathbf{Y}\mathbf{X}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left(\mathbf{1}_N^T \otimes \left(\mathbf{W} \odot \mathbf{P}^{yx}(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_{yx} \mathbf{e}_j \right) \right),$$

$$(2.51) \quad \partial_{\mathbf{W}\mathbf{X}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left(\mathbf{1}_N^T \otimes \left(\mathbf{P}^x(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_x \mathbf{e}_j \right) \right),$$

$$(2.52) \quad \partial_{\mathbf{X}\mathbf{Y}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left(\mathbf{1}_N^T \otimes \left(\mathbf{W} \odot \mathbf{P}^{xy}(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_{xy} \mathbf{e}_j \right) \right),$$

$$(2.53) \quad \partial_{\mathbf{Y}\mathbf{Y}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left(\mathbf{1}_N^T \otimes \left(\mathbf{W} \odot \mathbf{P}^{yy}(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_{yy} \mathbf{e}_j \right) \right),$$

$$(2.54) \quad \partial_{\mathbf{W}\mathbf{Y}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left(\mathbf{1}_N^T \otimes \left(\mathbf{P}^y(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_y \mathbf{e}_j \right) \right),$$

$$(2.55) \quad \partial_{\mathbf{X}\mathbf{W}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left(\mathbf{1}_N^T \otimes \left(\mathbf{P}^x(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_x \mathbf{e}_j \right) \right),$$

$$(2.56) \quad \partial_{\mathbf{Y}\mathbf{W}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = I_{N \times N} \odot \left(\mathbf{1}_N^T \otimes \left(\mathbf{P}^y(\mathbf{X}, \mathbf{Y})^T \mathcal{D}_y \mathbf{e}_j \right) \right),$$

$$(2.57) \quad \partial_{\mathbf{W}\mathbf{W}}(F_j(\mathbf{X}, \mathbf{Y}, \mathbf{W})) = 0_{N \times N},$$

where $\cdot \otimes \cdot$ symbolizes the Kronecker product, and $\cdot \odot \cdot$ is component-wise multiplication along the columns of the argument to the right⁵. We verify the correctness of our formulae for the first and second derivatives of our vector and scalar objectives by comparing with second-order finite difference approximations. In particular, we show in Figure 6 the relative error in the 2-norm between the analytical evaluation and the finite difference approximation. Since the difference scheme is second order, we expect the error to behave like $\mathcal{O}(h^2)$, where h is the stencil width. That is, we should observe that

$$\log(\|\mathcal{D}(\mathbf{F}) - \hat{\mathcal{D}}(\mathbf{F})\|) \approx 2 \log(h) + \beta,$$

where β is the order constant, and $\hat{\mathcal{D}}(\mathbf{F})$ is the finite difference approximation. The same relationship holds for $\mathcal{D}^2(F)$ and other derivatives as well. On a log-log scale, this means the slope of the error lines should be around 2. Indeed, we do observe this relationship in the regime of asymptotic convergence, before round-off error dominates truncation error for the finite difference scheme.

TODO: Derive the derivative and promotion operators for the general case of $d > 2$. UPDATE: this is done in the paper “A New Family of Orghogonal Polynomials in Three Variables, by Aktas, Area, and Guldogan” At this point, we can parse out what we mean by applying the inverse of the vector Hessian of $\mathbf{F} : \mathbb{R}^{3N} \rightarrow \mathbb{R}^M$ to the vector gradient in (2.18). We know that

$$\mathcal{D}^2(\mathbf{F}(\mathbf{z})) \in \mathbb{R}^{M \times 3N \times 3N}, \quad \mathcal{D}(\mathbf{F}(\mathbf{z})) \in \mathbb{R}^{M \times 3N}.$$

⁵Viewing $\mathbf{W}$ as a column of size N , we compute the component-wise product with columns of the matrix on the right. It would be more correct to define a function $\text{repmat}(W, k)$, which takes a vector $W \in \mathbb{R}^N$ and repeats it k times, producing a matrix in $\mathbb{R}^{N \times k}$ with each of the k columns a copy of W . However, for $N \neq M$, the ambiguity in matching sizes of the arguments to $\odot$ is eliminated.

Then, we take the following natural interpretation⁶

$$\begin{aligned} [\mathcal{D}^2(\mathbf{F}(\mathbf{z}))]^{-1} &:= [[\mathcal{D}^2(F_1(\mathbf{z}))]^{-1} \quad \cdots \quad [\mathcal{D}^2(F_M(\mathbf{z}))]^{-1}] \\ \nabla(\mathbf{F}(\mathbf{z})) &:= \begin{bmatrix} \mathcal{D}(F_1(\mathbf{z}))^T \\ \vdots \\ \mathcal{D}(F_M(\mathbf{z}))^T \end{bmatrix} \\ \Rightarrow [\mathcal{D}^2(\mathbf{F}(\mathbf{z}))]^{-1} \mathcal{D}(\mathbf{F}(\mathbf{z})) &:= \sum_{j=1}^M \mathcal{D}^2(F_j(\mathbf{z}))^{-1} \nabla(F_j(\mathbf{z})) \in \mathbb{R}^{3N \times 1}, \quad (\text{as required}) \end{aligned}$$

Derivative of the Dunkl–Xu Simplex Basis. We begin with the d -simplex orthogonal polynomials defined in Dunkl–Xu (Eq. 2.5.21). Let $x = (x_1, \dots, x_d)$ satisfy

$$x_j \geq 0, \quad x_1 + \cdots + x_d \leq 1,$$

and let $\alpha = (\alpha_1, \dots, \alpha_d) \in \mathbb{N}_0^d$. Define the prefix sums

$$s_j(x) := \sum_{i=1}^{j-1} x_i, \quad \omega_j(x) := 1 - s_j(x), \quad t_j(x) := \frac{2x_j}{\omega_j(x)} - 1,$$

where by convention $s_1(x) = 0$. Let

$$J_j(x) := P_{\alpha_j}^{(a_j, \kappa_j - \frac{1}{2})}(t_j(x)),$$

where a_j and κ_j are the parameters appearing in (2.5.21), and $P_n^{(a,b)}$ denotes the Jacobi polynomial on $[-1, 1]$. Then (2.5.21) states that

$$P_\alpha(x) = [h_\alpha]^{-1} \prod_{j=1}^d (\omega_j(x)^{\alpha_j} J_j(x)),$$

where h_α is a constant depending only on α and the Jacobi parameters.

Define

$$\Phi_j(x) := \omega_j(x)^{\alpha_j} J_j(x), \quad j = 1, \dots, d,$$

so that

$$P_\alpha(x) = [h_\alpha]^{-1} \prod_{j=1}^d \Phi_j(x).$$

Step 1. Logarithmic differentiation of the product. Let

$$F(x) := \prod_{j=1}^d \Phi_j(x).$$

Then

$$\log F(x) = \sum_{j=1}^d \log \Phi_j(x),$$

and by differentiating,

$$\frac{1}{F(x)} \frac{\partial F}{\partial x_\ell} = \sum_{j=1}^d \frac{\partial}{\partial x_\ell} \log \Phi_j(x).$$

Multiplying by $F(x)$ gives the product rule

$$\boxed{\frac{\partial}{\partial x_\ell} \left(\prod_{j=1}^d \Phi_j(x) \right) = \left(\prod_{j=1}^d \Phi_j(x) \right) \sum_{j=1}^d \frac{\partial}{\partial x_\ell} \log \Phi_j(x).}$$

Hence

$$\frac{\partial P_\alpha}{\partial x_\ell}(x) = P_\alpha(x) \sum_{j=1}^d \frac{\partial}{\partial x_\ell} \log \Phi_j(x).$$

⁶For a function $\mathbf{u} : \mathbb{R}^N \rightarrow \mathbb{R}^M$, we note the distinction between $\mathcal{D}(\mathbf{u}(\mathbf{z}))$ and $\nabla(\mathbf{u}(\mathbf{z}))$. The former is an element of $L(\mathbb{R}^N, \mathbb{R}^M)$, equivalently viewed as a set of M elements of $L(\mathbb{R}^N, \mathbb{R})$, or M row vectors in $\mathbb{R}^N$ (left argument of dot product). The latter evaluated at a point is as an element of $L(\mathbb{R}^M, \mathbb{R}^N)$, equivalently viewed as a set of M elements of $L(\mathbb{R}, \mathbb{R}^N)$, or M column vectors in $\mathbb{R}^N$ (right argument of dot product). TLDR; $[\mathcal{D}(\mathbf{u})]^T = \nabla(\mathbf{u})$

Step 2. Derivative of the factor $\log(\omega_j^{\alpha_j})$. Since

$$\omega_j(x) = 1 - s_j(x), \quad s_j(x) = \sum_{i=1}^{j-1} x_i,$$

we have

$$\frac{\partial s_j}{\partial x_\ell} = \begin{cases} 1, & 1 \leq \ell \leq j-1, \\ 0, & \ell \geq j. \end{cases}$$

Thus

$$\frac{\partial \omega_j}{\partial x_\ell} = -\frac{\partial s_j}{\partial x_\ell} = \begin{cases} -1, & \ell \leq j-1, \\ 0, & \ell \geq j. \end{cases}$$

If $\alpha_j > 0$, then

$$\omega_j(x)^{\alpha_j} = \exp(\alpha_j \log \omega_j(x)),$$

and therefore

$$\frac{\partial}{\partial x_\ell} \log(\omega_j^{\alpha_j}) = \alpha_j \frac{1}{\omega_j} \frac{\partial \omega_j}{\partial x_\ell} = \begin{cases} -\frac{\alpha_j}{\omega_j(x)}, & \ell \leq j-1, \\ 0, & \ell \geq j. \end{cases}$$

Step 3. Derivative of $t_j(x)$. Recall

$$t_j(x) = \frac{2x_j}{\omega_j(x)} - 1.$$

We compute:

$$\frac{\partial t_j}{\partial x_\ell} = \begin{cases} \frac{2}{\omega_j(x)}, & \ell = j, \\ \frac{2x_j}{\omega_j(x)^2}, & 1 \leq \ell \leq j-1, \\ 0, & \ell > j. \end{cases}$$

Step 4. Derivative of the Jacobi factor $J_j(x)$. For $\alpha_j > 0$, define

$$J_j(x) = P_{\alpha_j}^{(a_j, b_j)}(t_j(x)), \quad b_j := \kappa_j - \frac{1}{2}.$$

The standard Jacobi derivative identity is

$$\frac{d}{dt} P_n^{(a,b)}(t) = \frac{1}{2}(n+a+b+1) P_{n-1}^{(a+1, b+1)}(t), \quad n \geq 1.$$

Hence

$$\frac{\partial J_j}{\partial x_\ell} = \frac{1}{2}(\alpha_j + a_j + b_j + 1) P_{\alpha_j-1}^{(a_j+1, b_j+1)}(t_j(x)) \frac{\partial t_j}{\partial x_\ell}.$$

Thus

$$\frac{\partial}{\partial x_\ell} \log J_j(x) = \frac{1}{J_j(x)} \frac{\partial J_j}{\partial x_\ell} = R_j(x) \frac{\partial t_j}{\partial x_\ell},$$

where

$$R_j(x) := \frac{\frac{1}{2}(\alpha_j + a_j + b_j + 1) P_{\alpha_j-1}^{(a_j+1, b_j+1)}(t_j(x))}{P_{\alpha_j}^{(a_j, b_j)}(t_j(x))},$$

and $R_j(x) = 0$ if $\alpha_j = 0$.

Step 5. Final derivative formula. Combining all pieces,

$$\boxed{\frac{\partial P_\alpha}{\partial x_\ell}(x) = P_\alpha(x) \sum_{j=1}^d \left[-\mathbf{1}_{\{\ell \leq j-1\}} \frac{\alpha_j}{\omega_j(x)} + R_j(x) \frac{\partial t_j}{\partial x_\ell}(x) \right].}$$

Here $\omega_j(x)$, $t_j(x)$, $R_j(x)$, and $\frac{\partial t_j}{\partial x_\ell}(x)$ are given explicitly above. This formula holds for the full Dunkl–Xu orthogonal basis on the d -simplex defined in Eq. (2.5.21).

Derivative Operators for Dunkl–Xu Simplex Polynomials (General D). We work on the D -simplex

$$T_D = \{x \in \mathbb{R}^D : x_i \geq 0, \quad |x| := x_1 + \cdots + x_D \leq 1\}.$$

The Jacobi weight is

$$w_\kappa(x) = x_1^{\kappa_1-1/2} \cdots x_D^{\kappa_D-1/2} (1 - |x|)^{\kappa_{D+1}-1/2}, \quad \kappa_i > -\frac{1}{2}.$$

1. Dunkl–Xu basis (Equation 2.5.21). Let $\alpha = (\alpha_1, \dots, \alpha_D) \in \mathbb{N}_0^D$. Define

$$s_j(x) := x_1 + \cdots + x_{j-1}, \quad \omega_j(x) := 1 - s_j(x), \quad t_j(x) := \frac{2x_j}{\omega_j(x)} - 1,$$

with $s_1(x) := 0$. Jacobi parameters at level j are

$$a_j(\alpha, \kappa) := 2|\alpha^{j+1}| + |\kappa^{j+1}| + \frac{1}{2}(D - j - 1), \quad b_j := \kappa_j - \frac{1}{2}.$$

We take the (already normalized) Dunkl–Xu basis

$$P_\alpha^{(\kappa)}(x) := \prod_{j=1}^D \omega_j(x)^{\alpha_j} P_{\alpha_j}^{(a_j(\alpha, \kappa), b_j)}(t_j(x)),$$

with normalization chosen so that

$$\langle P_\alpha^{(\kappa)}, P_\beta^{(\kappa)} \rangle_{w_\kappa} = \delta_{\alpha\beta}, \quad \langle f, g \rangle_{w_\kappa} := \int_{T_D} f(x) g(x) w_\kappa(x) dx.$$

(Your implementation achieves this via the precomputed factors `inv_h_alpha`.)

2. Pointwise derivative formula. Introduce

$$\Phi_j(x) := \omega_j(x)^{\alpha_j} P_{\alpha_j}^{(a_j(\alpha, \kappa), b_j)}(t_j(x)),$$

so that

$$P_\alpha^{(\kappa)}(x) = \prod_{j=1}^D \Phi_j(x).$$

Logarithmic differentiation yields

$$(2.58) \quad \frac{\partial P_\alpha^{(\kappa)}}{\partial x_\ell}(x) = P_\alpha^{(\kappa)}(x) \sum_{j=1}^D \frac{\partial}{\partial x_\ell} \log \Phi_j(x).$$

We compute, for $\alpha_j > 0$,

$$\frac{\partial}{\partial x_\ell} \log(\omega_j^{\alpha_j}) = \begin{cases} -\frac{\alpha_j}{\omega_j(x)}, & \ell \leq j-1, \\ 0, & \ell \geq j, \end{cases}$$

and using the Jacobi identity

$$\frac{d}{dt} P_n^{(a,b)}(t) = \frac{n+a+b+1}{2} P_{n-1}^{(a+1,b+1)}(t),$$

we obtain

$$\frac{\partial}{\partial x_\ell} \log P_{\alpha_j}^{(a_j, b_j)}(t_j) = R_j(x) \frac{\partial t_j}{\partial x_\ell}(x),$$

where, for $\alpha_j > 0$,

$$R_j(x) := \frac{\frac{1}{2}(\alpha_j + a_j + b_j + 1) P_{\alpha_j-1}^{(a_j+1, b_j+1)}(t_j(x))}{P_{\alpha_j}^{(a_j, b_j)}(t_j(x))}.$$

The derivative of t_j is

$$\frac{\partial t_j}{\partial x_\ell}(x) = \begin{cases} \frac{2}{\omega_j(x)}, & \ell = j, \\ \frac{2x_j}{\omega_j(x)^2}, & 1 \leq \ell \leq j-1, \\ 0, & \ell > j. \end{cases}$$

Plugging these into (2.58) yields the compact general- D formula:

$$(2.59) \quad \boxed{\frac{\partial P_\alpha^{(\kappa)}}{\partial x_\ell}(x) = P_\alpha^{(\kappa)}(x) \sum_{j=1}^D \left[-\mathbf{1}_{(\ell \leq j-1)} \frac{\alpha_j}{\omega_j(x)} + R_j(x) \frac{\partial t_j}{\partial x_\ell}(x) \right]}$$

3. Promoted parameter sets. Differentiating the Jacobi factor at level j transforms

$$(a_j, b_j) \mapsto (a_j + 1, b_j + 1),$$

which corresponds to shifting the tail of κ :

$$(2.60) \quad \kappa_r^{(j)} := \begin{cases} \kappa_r, & r < j, \\ \kappa_r + \frac{1}{2}, & r \geq j. \end{cases}$$

Thus each level- j contribution to the derivative naturally lies in $\Pi_{|\alpha|-1}^D(\kappa^{(j)})$, the degree- $|\alpha| - 1$ space for the promoted parameter set $\kappa^{(j)}$.

4. Selection rule. Consider the inner product

$$I_{\beta, \alpha}^{(\ell, j)} := \left\langle \frac{\partial}{\partial x_\ell} P_\alpha^{(\kappa)}, P_\beta^{(\kappa^{(j)})} \right\rangle_{w_{\kappa^{(j)}}},$$

for some multi-index β with $|\beta| \leq |\alpha| - 1$.

The polynomial $P_\alpha^{(\kappa)}$ is a product over levels $m = 1, \dots, D$ of 1D Jacobi factors in the transformed coordinates (t_m, ω_m) . The derivative ∂_{x_ℓ} in (2.59) is a sum over $j = 1, \dots, D$, where the j th summand modifies only the j th factor, lowering its 1D Jacobi degree by 1 while leaving all other 1D degrees unchanged.

On the other hand, for fixed promoted parameter set $\kappa^{(j)}$, the family $\{P_\beta^{(\kappa^{(j)})}\}$ is orthonormal and separates by 1D degree at each level. Consequently, the inner product $I_{\beta, \alpha}^{(\ell, j)}$ is zero unless β matches α in every unaffected coordinate and is lowered by 1 at the j th coordinate.

More precisely,

$$(2.61) \quad I_{\beta, \alpha}^{(\ell, j)} \neq 0 \implies \beta = \alpha - e_j, \quad j \geq \ell, \quad \alpha_j > 0,$$

where e_j is the j th standard basis vector in $\mathbb{R}^D$. This is the *selection rule*: differentiating a Dunkl–Xu basis polynomial can only lower one Jacobi index α_j at a time, and only for those $j \geq \ell$ such that ∂_{x_ℓ} affects the j th factor.

5. Sparse coefficient mapping. Expand the derivative in the promoted bases $\{P_\beta^{(\kappa^{(j)})}\}$:

$$(2.62) \quad \frac{\partial}{\partial x_\ell} P_\alpha^{(\kappa)}(x) = \sum_{j=\ell}^D c_\alpha^{(\ell, j)}(\kappa) P_{\alpha - e_j}^{(\kappa^{(j)})}(x),$$

where each term is present only if $\alpha_j > 0$, and

$$(2.63) \quad c_\alpha^{(\ell, j)}(\kappa) := \left\langle \frac{\partial}{\partial x_\ell} P_\alpha^{(\kappa)}, P_{\alpha - e_j}^{(\kappa^{(j)})} \right\rangle_{w_{\kappa^{(j)}}}.$$

6. Sparse derivative matrices. Let Π_n^D denote polynomials of total degree $\leq n$. For each ℓ define the block operator

$$D^{(\ell)} = \begin{bmatrix} D^{(\ell, \ell)} \\ D^{(\ell, \ell+1)} \\ \vdots \\ D^{(\ell, D)} \end{bmatrix} : \mathbb{R}^{\dim \Pi_n^D} \longrightarrow \bigoplus_{j=\ell}^D \mathbb{R}^{\dim \Pi_{n-1}^D},$$

where each block satisfies

$$(2.64) \quad (D^{(\ell, j)})_{\alpha - e_j, \alpha} = c_\alpha^{(\ell, j)}(\kappa), \quad \alpha_j > 0,$$

and all other entries vanish.

Thus for any modal expansion

$$u(x) = \sum_{|\alpha| \leq n} c_\alpha P_\alpha^{(\kappa)}(x),$$

its derivative is

$$\frac{\partial u}{\partial x_\ell}(x) = \sum_{j=\ell}^D \sum_{|\alpha| \leq n} c_\alpha c_\alpha^{(\ell, j)}(\kappa) P_{\alpha - e_j}^{(\kappa^{(j)})}(x),$$

which is an extremely sparse operation in coefficient space.

Limiting values of Jacobi polynomials on simplex faces (general D). Let

$$T^D := \{x \in \mathbb{R}^D : x_i \geq 0, x_1 + \cdots + x_D \leq 1\}$$

be the reference D -simplex. For parameters $\kappa = (\kappa_1, \dots, \kappa_{D+1})$ with $\kappa_i > -\frac{1}{2}$ and multi-index $\alpha = (\alpha_1, \dots, \alpha_D) \in \mathbb{N}_0^D$, the Koornwinder–Jacobi polynomial $P_\alpha^\kappa(x)$ on T^D admits the product representation (Dunkl–Xu, (2.5.21))

$$(2.65) \quad P_\alpha^\kappa(x) = c_\alpha^\kappa \prod_{\ell=1}^D \left(1 - |x^{\ell-1}|\right)^{\alpha_\ell} P_{\alpha_\ell}^{(a_\ell, b_\ell)}\left(\frac{2x_\ell}{1 - |x^{\ell-1}|} - 1\right), \quad x \in T^D,$$

where

$$|x^{\ell-1}| := x_1 + \cdots + x_{\ell-1}, \quad |x^0| := 0,$$

and

$$(2.66) \quad a_\ell = 2|\alpha^{\ell+1}| + |\kappa^{\ell+1}| + \frac{1}{2}(D - \ell - 1), \quad b_\ell = \kappa_\ell - \frac{1}{2},$$

with tail notation $\alpha^{\ell+1} = (\alpha_{\ell+1}, \dots, \alpha_D)$ and $\kappa^{\ell+1} = (\kappa_{\ell+1}, \dots, \kappa_{D+1})$. The normalisation c_α^κ is independent of x .

Approaching a boundary face. We consider a path $x(t) \in T^D$ for $t > 0$ such that

$$\lim_{t \downarrow 0} x(t) = x^*,$$

where x^* lies on a boundary face of T^D . For each coordinate index ℓ define

$$s_\ell(t) := x_1(t) + \cdots + x_{\ell-1}(t), \quad y_\ell(t) := x_\ell(t).$$

A face is characterised by the behaviour of the partial sums $s_\ell(t)$: whenever $s_\ell(t) \rightarrow 1$, the remaining coordinates $x_\ell(t), \dots, x_D(t)$ necessarily tend to zero (the simplex constraint), identifying the corresponding codimension- $(\ell - 1)$ face of T^D .

Define the ℓ th factor in (2.65) as

$$(2.67) \quad \Phi_\ell(t) := (1 - s_\ell(t))^{\alpha_\ell} P_{\alpha_\ell}^{(a_\ell, b_\ell)}\left(\frac{2y_\ell(t)}{1 - s_\ell(t)} - 1\right).$$

Then

$$P_\alpha^\kappa(x(t)) = c_\alpha^\kappa \prod_{\ell=1}^D \Phi_\ell(t).$$

To determine the limiting value on the face, it suffices to compute

$$L_\ell := \lim_{t \downarrow 0} \Phi_\ell(t)$$

in the case when $s_\ell(t) \rightarrow 1$. If $s_\ell(t) \not\rightarrow 1$, the denominator stays bounded away from zero and the limit is just the ordinary evaluation $\Phi_\ell(0)$, so no special treatment is required. Thus we now focus on the singular case $s_\ell(t) \rightarrow 1$.

Singular limit of a single factor. Fix an index ℓ for which $s_\ell(t) \rightarrow 1$ and write

$$s := s_\ell(t), \quad y := y_\ell(t), \quad n := \alpha_\ell, \quad a := a_\ell, \quad b := b_\ell.$$

Then

$$(2.68) \quad \Phi_\ell(s, y) = (1 - s)^n P_n^{(a, b)}\left(\frac{2y}{1 - s} - 1\right).$$

A standard identity for Jacobi polynomials gives the leading coefficient

$$(2.69) \quad [q^n] P_n^{(a, b)}(q) = \frac{1}{2^n} \binom{2n + a + b}{n},$$

so as $|q| \rightarrow \infty$,

$$(2.70) \quad P_n^{(a, b)}(q) = \frac{1}{2^n} \binom{2n + a + b}{n} q^n + \sum_{r=0}^{n-1} c_r q^r, \quad c_r \in \mathbb{R}.$$

In (2.68) the Jacobi argument is

$$q(s, y) := \frac{2y}{1 - s} - 1,$$

and for fixed y one has $|q(s, y)| \rightarrow \infty$ as $s \rightarrow 1^-$. Substituting (2.70) into (2.68),

$$(2.71) \quad \begin{aligned} \Phi_\ell(s, y) &= (1-s)^n \left[\frac{1}{2^n} \binom{2n+a+b}{n} q(s, y)^n + \sum_{r=0}^{n-1} c_r q(s, y)^r \right] \\ &= \frac{1}{2^n} \binom{2n+a+b}{n} (1-s)^n q(s, y)^n + \sum_{r=0}^{n-1} c_r (1-s)^n q(s, y)^r. \end{aligned}$$

Since $q(s, y) \sim 2y/(1-s)$ as $s \rightarrow 1^-$, we obtain

$$(1-s)^n q(s, y)^n \sim (2y)^n, \quad (1-s)^n q(s, y)^r \sim (1-s)^{n-r} (2y)^r \rightarrow 0, \quad r < n.$$

Thus all lower-order contributions vanish, yielding the singular limit

$$(2.72) \quad \lim_{s \rightarrow 1^-} \Phi_\ell(s, y) = \frac{1}{2^{\alpha_\ell}} \binom{2\alpha_\ell + a_\ell + b_\ell}{\alpha_\ell} (2x_\ell^*)^{\alpha_\ell},$$

where $x_\ell^* = \lim_{t \downarrow 0} x_\ell(t)$ is the coordinate value of the face point x^* .

Limit of the full polynomial. Let $x^* = \lim_{t \downarrow 0} x(t)$ denote the limit point on a boundary face, and write $s_\ell^* := \lim_{t \downarrow 0} s_\ell(t)$ for the corresponding partial sums. For each ℓ with $s_\ell^* < 1$ the factor $\Phi_\ell(t)$ is continuous at $t = 0$ and we simply have $\lim_{t \downarrow 0} \Phi_\ell(t) = \Phi_\ell(x^*)$. For each ℓ with $s_\ell^* = 1$ the singular limit (2.72) applies. Collecting all factors, we obtain

$$(2.73) \quad \lim_{t \downarrow 0} P_\alpha^\kappa(x(t)) = c_\alpha^\kappa \prod_{\substack{\ell=1 \\ s_\ell^* < 1}}^D \Phi_\ell(x^*) \cdot \prod_{\substack{\ell=1 \\ s_\ell^* = 1}}^D \left[\frac{1}{2^{\alpha_\ell}} \binom{2\alpha_\ell + a_\ell + b_\ell}{\alpha_\ell} (2x_\ell^*)^{\alpha_\ell} \right],$$

where, for each ℓ ,

$$\Phi_\ell(x^*) = (1-s_\ell^*)^{\alpha_\ell} P_{\alpha_\ell}^{(a_\ell, b_\ell)} \left(\frac{2x_\ell^*}{1-s_\ell^*} - 1 \right), \quad s_\ell^* = \sum_{i=1}^{\ell-1} x_i^*.$$

In particular, for $D = 2$ and $s_2 = x_1 \rightarrow 1$ this recovers the classical vertex limit used in (1.5). For $D = 3$, singular limits appear along edges and triangular faces whenever $s_\ell(t) \rightarrow 1$, and (2.72) provides the correct replacement for the corresponding factor in (2.65).

3. Mapped tensor-product quadrature on the simplex.

3.1. Reference simplex, barycentric coordinates, and Dirichlet–Jacobi weights. Let the reference d -simplex be

$$\Delta_d = \left\{ x \in \mathbb{R}^d : x_i \geq 0, \sum_{i=1}^d x_i \leq 1 \right\}.$$

Introduce barycentric coordinates $\lambda = (\lambda_1, \dots, \lambda_{d+1})$ with

$$\lambda_i = x_i \quad (1 \leq i \leq d), \quad \lambda_{d+1} = 1 - \sum_{i=1}^d x_i, \quad \sum_{i=1}^{d+1} \lambda_i = 1, \quad \lambda_i \geq 0.$$

For parameters $\kappa \in \mathbb{R}^{d+1}$ with $\kappa_i > 0$ we use the Dirichlet–Jacobi weight

$$(3.1) \quad w_\kappa(x) = C_\kappa \prod_{i=1}^{d+1} \lambda_i(x)^{\kappa_i - \frac{1}{2}}, \quad C_\kappa^{-1} = \int_{\Delta_d} \prod_{i=1}^{d+1} \lambda_i(x)^{\kappa_i - \frac{1}{2}} dx,$$

and define the orthonormal simplex polynomials $\{\phi_\alpha^{(\kappa)}\}$ by

$$\langle \phi_\alpha^{(\kappa)}, \phi_\beta^{(\kappa)} \rangle_{L^2(\Delta_d, w_\kappa)} = \delta_{\alpha\beta}, \quad |\alpha| \leq n.$$

3.2. Stick-breaking map to $[0, 1]^d$. A convenient way to build quadrature nodes on Δ_d is via the stick-breaking parameterization. Let $t = (t_1, \dots, t_d) \in [0, 1]^d$ and define

$$(3.2) \quad \begin{aligned} \lambda_1 &= t_1, \\ \lambda_2 &= (1-t_1)t_2, \\ \lambda_3 &= (1-t_1)(1-t_2)t_3, \\ &\vdots \\ \lambda_d &= \left(\prod_{j=1}^{d-1} (1-t_j) \right) t_d, \\ \lambda_{d+1} &= \prod_{j=1}^d (1-t_j), \end{aligned}$$

so that $x_i = \lambda_i$ for $i \leq d$. This map is bijective between $[0, 1]^d$ and the interior of Δ_d .

3.3. Jacobian determinant. The Jacobian of $x(t)$ (equivalently of $\lambda(t)$ restricted to the first d components) is lower-triangular in a suitable ordering, and its determinant is

$$(3.3) \quad \left| \frac{\partial x}{\partial t} \right| = \prod_{j=1}^d (1 - t_j)^{d-j}.$$

3.4. Factorization of the simplex measure into 1D Jacobi weights. Substitute $\lambda(t)$ into the Dirichlet weight. From (3.1) and (3.2) we have

$$\prod_{i=1}^d \lambda_i^{\kappa_i - \frac{1}{2}} = \prod_{j=1}^d \left(t_j^{\kappa_j - \frac{1}{2}} \prod_{m=1}^{j-1} (1 - t_m)^{\kappa_j - \frac{1}{2}} \right), \quad \lambda_{d+1}^{\kappa_{d+1} - \frac{1}{2}} = \prod_{m=1}^d (1 - t_m)^{\kappa_{d+1} - \frac{1}{2}}.$$

Collecting all powers of each $(1 - t_j)$ from: (i) the later barycentric coordinates $\lambda_{j+1}, \dots, \lambda_d$, (ii) λ_{d+1} , and (iii) the Jacobian (3.3), one obtains a complete factorization:

$$(3.4) \quad w_\kappa(x(t)) \left| \frac{\partial x}{\partial t} \right| = \tilde{C}_\kappa \prod_{j=1}^d \left[t_j^{\kappa_j - \frac{1}{2}} (1 - t_j)^{\eta_j - \frac{1}{2}} \right],$$

where the effective “right” Jacobi parameters are

$$(3.5) \quad \eta_j := \sum_{m=j+1}^{d+1} \kappa_m + (d - j).$$

Thus the weighted simplex integral becomes a tensor-product of 1D Jacobi-weighted integrals:

$$(3.6) \quad \int_{\Delta_d} f(x) w_\kappa(x) dx = \int_{[0,1]^d} f(x(t)) \prod_{j=1}^d t_j^{\kappa_j - \frac{1}{2}} (1 - t_j)^{\eta_j - \frac{1}{2}} dt.$$

3.5. 1D Gauss–Jacobi rules on $[0, 1]$. For each axis j , define a Gauss–Jacobi quadrature on $[0, 1]$ for the weight

$$\rho_j(t) := t^{\kappa_j - \frac{1}{2}} (1 - t)^{\eta_j - \frac{1}{2}}.$$

Let $\{(t_q^{(j)}, \omega_q^{(j)})\}_{q=1}^Q$ integrate polynomials up to degree $2Q - 1$ against ρ_j . (Equivalently, one may map the standard $[-1, 1]$ Gauss–Jacobi nodes via $t = (1 + s)/2$ with the usual weight scaling.)

3.6. Mapped tensor-product quadrature on Δ_d . Form the tensor product rule on $[0, 1]^d$:

$$t^{(q)} = (t_{q_1}^{(1)}, \dots, t_{q_d}^{(d)}), \quad \Omega^{(q)} = \prod_{j=1}^d \omega_{q_j}^{(j)}.$$

Map nodes to the simplex by $x^{(q)} = x(t^{(q)})$ via (3.2). Then for smooth f ,

$$(3.7) \quad \int_{\Delta_d} f(x) w_\kappa(x) dx \approx \sum_q \Omega^{(q)} f(x^{(q)}).$$

Because the factorization (3.4) already includes the Dirichlet–Jacobi weight, this rule is tailored for accurate evaluation of inner products in $L^2(\Delta_d, w_\kappa)$.

Parameter selection per axis.. The stick-breaking map induces the 1D Jacobi weights $\rho_j(t)$ with parameters

$$\alpha_j = \kappa_j - \frac{1}{2}, \quad \beta_j = \eta_j - \frac{1}{2}, \quad \eta_j = \sum_{m=j+1}^{d+1} \kappa_m + (d - j),$$

so the quadrature construction is entirely determined by κ and d .

Notes on degree of exactness.. If f is a polynomial in barycentric coordinates and the basis functions are polynomial, then (3.7) integrates exactly up to a degree determined by the 1D orders Q . In practice we choose Q so that all inner products defining operator entries are resolved to machine precision, and then prune near-zeros consistent with the expected sparsity pattern.

4. Promotion and differentiation operators from quadrature.

4.1. Promotion matrices. For degree $\leq n$, let $M = \dim \Pi_n(\Delta_d)$. Given parameter vectors κ, κ' , define the promotion matrix

$$(4.1) \quad (K_{\kappa \rightarrow \kappa'})_{ij} := \langle \phi_i^{(\kappa')}, \phi_j^{(\kappa)} \rangle_{L^2(\Delta_d, w_{\kappa'})} = \int_{\Delta_d} \phi_i^{(\kappa')}(x) \phi_j^{(\kappa)}(x) w_{\kappa'}(x) dx.$$

Using mapped quadrature with weight $w_{\kappa'}$:

$$(4.2) \quad K_{\kappa \rightarrow \kappa'} \approx V(\kappa')^\top \text{diag}(\omega) V(\kappa),$$

where $V(\kappa)_{qj} = \phi_j^{(\kappa)}(x^{(q)})$ and ω are quadrature weights for $w_{\kappa'}$.

4.2. Differentiation matrices and the natural parameter shift. Define the coefficient differentiation matrix for ∂_{x_i} :

$$(4.3) \quad (D_{i;\kappa \rightarrow \kappa'})_{ab} := \langle \phi_a^{(\kappa')}, \partial_{x_i} \phi_b^{(\kappa)} \rangle_{L^2(\Delta_d, w_{\kappa'})}.$$

Empirically (and consistent with known low-dimensional sparse recurrences), the sparsest stable mapping is obtained by the *natural shift*

$$(4.4) \quad \kappa' = \kappa + \delta^{(i)}, \quad \delta^{(i)} = e_i + e_{d+1},$$

i.e. increment the i -th and the last Dirichlet parameter.

Quadrature evaluation using analytic gradients.. We evaluate (4.3) via mapped quadrature with weight $w_{\kappa'}$:

$$(4.5) \quad D_{i;\kappa \rightarrow \kappa'} \approx V(\kappa')^\top \text{diag}(\omega) (\partial_{x_i} V(\kappa)),$$

where $\partial_{x_i} V(\kappa)$ is obtained by evaluating analytic gradients of the orthonormal basis at the quadrature nodes. After dense evaluation, we apply structured pruning based on row-relative thresholds and an absolute floor, yielding the observed banded/block-banded sparsity patterns in graded total-degree ordering.

4.3. Promoted Laplacian on coefficients. Second derivatives are composed in coefficient space:

$$D_{ii}(\kappa) := D_{i;\kappa + \delta^{(i)} \rightarrow \kappa + 2\delta^{(i)}} D_{i;\kappa \rightarrow \kappa + \delta^{(i)}}.$$

Each D_{ii} lives in parameter space $\kappa + 2\delta^{(i)}$. To sum, promote to a common target

$$\kappa_\Delta := \kappa + 2\mathbf{1}, \quad \mathbf{1} = (1, \dots, 1) \in \mathbb{R}^{d+1}.$$

Define the promoted Laplacian

$$(4.6) \quad A_{\Delta, \kappa} := \sum_{i=1}^d K_{\kappa + 2\delta^{(i)} \rightarrow \kappa_\Delta} D_{ii}(\kappa),$$

mapping $\Pi_n^{(\kappa)}$ coefficients to $\Pi_n^{(\kappa_\Delta)}$ coefficients of Δu .

5. Poisson on one tetrahedron via nullspace completion.

5.1. Discrete formulation. On $K = \Delta_3$ (reference tetrahedron), solve

$$-\Delta u = f \text{ in } K, \quad u = g \text{ on } \partial K,$$

with $u_h(x) = \sum_{j=1}^M c_j \phi_j^{(\kappa)}(x)$ and $M = \dim \Pi_n(K)$.

5.2. Interior constraints (strong operator in coefficient space). Since $\Delta \Pi_n \subset \Pi_{n-2}$, only $M_{\text{int}} := \dim \Pi_{n-2}(K)$ constraints are active. Let $A_{\Delta, \kappa}$ be the promoted Laplacian (4.6) and define

$$L_{\text{int}} := A_{\Delta, \kappa} [1:M_{\text{int}}, :] \in \mathbb{R}^{M_{\text{int}} \times M}.$$

Project f into the κ_Δ basis:

$$(f_\Delta)_a := \langle \phi_a^{(\kappa_\Delta)}, f \rangle_{L^2(K, w_{\kappa_\Delta})}.$$

Then the interior equation is

$$(5.1) \quad L_{\text{int}} c = -f_{\text{int}}, \quad f_{\text{int}} := (f_\Delta)_{1:M_{\text{int}}}.$$

5.3. Boundary moments (weak Dirichlet enforcement). On each face F we choose an independent triangle basis (Option A) $\{\psi_\ell^{(F)}\}$ of degree $\leq n$. Stacking all faces yields N_b^{full} boundary test functions $\{\psi_\ell\}$. Define

$$(5.2) \quad (T_{\text{full}})_{\ell j} := \int_{\partial K} \psi_\ell(x) \phi_j^{(\kappa)}(x) dS, \quad (g_{\text{full}})_\ell := \int_{\partial K} \psi_\ell(x) g(x) dS,$$

assembled accurately by mapped triangle quadrature on each face.

5.4. Harmonic dimension and nullspace completion. Let $M = \dim \Pi_n(K)$ and $M_{\text{int}} = \dim \Pi_{n-2}(K)$. Then

$$k := M - M_{\text{int}} = (n+1)^2.$$

This k is the dimension of the discrete nullspace of L_{int} (harmonic modes). We solve (5.1)–(5.2) by:

1. Find a particular interior solution c_p satisfying $L_{\text{int}} c_p = -f_{\text{int}}$ (via a direct square solve on pivot columns of L_{int}).
2. Compute a basis $N \in \mathbb{R}^{M \times k}$ for $\ker(L_{\text{int}})$ (e.g. from an SVD).
3. Write all interior solutions as $c = c_p + Ny$ and impose Dirichlet:

$$T_{\text{full}}(c_p + Ny) = g_{\text{full}} \quad \Rightarrow \quad (T_{\text{full}} N)y = g_{\text{full}} - T_{\text{full}} c_p.$$

4. Select k independent rows of $(T_{\text{full}} N)$ by pivoted QR on $(T_{\text{full}} N)^\top$, solve the resulting $k \times k$ system for y , and recover c . For polynomial manufactured solutions, the method converges spectrally in n and reaches machine precision once n exceeds the solution degree.

6. Two tetrahedra: interface coupling and the local DtN map.

6.1. Geometry and interface notation. Let K_1 and K_2 be two tetrahedra sharing a common face $F = K_1 \cap K_2$. Let $\partial K_e = F \cup \Gamma_e$ where Γ_e denotes the external faces of K_e . We orient normals so that n_1 points out of K_1 and $n_2 = -n_1$ points out of K_2 on the shared face.

6.2. Face trace and flux coefficients. Choose a face basis on F (triangle basis of degree $\leq n$) and represent:

$$\hat{u}_F \in \mathbb{R}^{N_F}, \quad \hat{q}_F \in \mathbb{R}^{N_F},$$

where $\hat{u}_F$ are Dirichlet trace moment coefficients and $\hat{q}_F$ are flux moment coefficients, defined via (for each face test function ψ_ℓ on F)

$$(\hat{u}_F)_\ell \approx \int_F \psi_\ell u \, dS, \quad (\hat{q}_F)_\ell \approx \int_F \psi_\ell (\nabla u \cdot n) \, dS.$$

Similarly define external-face traces $\hat{u}_{\Gamma_e}$ from prescribed boundary data.

6.3. Element equations with boundary forcing split. On each element K_e we have:

$$L_{\text{int},e} c_e = -f_{\text{int},e}, \quad T_{e,\Gamma} c_e = \hat{u}_{\Gamma_e}, \quad T_{e,F} c_e = \hat{u}_F.$$

Here $T_{e,F}$ and $T_{e,\Gamma}$ are the appropriate blocks of the full boundary moment map $T_{\text{full},e}$ restricted to F and to Γ_e .

6.4. Element DtN map on the interface. Eliminate the volume coefficients c_e to obtain a map from interface trace to interface flux:

$$(6.1) \quad \hat{q}_F^{(e)} = S_e \hat{u}_F + \hat{q}_{F,f}^{(e)},$$

where S_e is the (local) Dirichlet-to-Neumann operator on the shared face and $\hat{q}_{F,f}^{(e)}$ is the forcing contribution (including prescribed external boundary data on Γ_e). In our nullspace-completion framework, (6.1) is obtained by: (i) computing c_p and N for $L_{\text{int},e}$, (ii) expressing the dependence of c_e on $\hat{u}_F$ through boundary moment constraints, and (iii) applying the face flux map $F_{e,F}$ to c_e .

6.5. Element DtN map on the interface (detailed elimination). Fix an element K_e with boundary $\partial K_e = F \cup \Gamma_e$, where F is a designated interface face (shared with a neighbor) and Γ_e denotes the remaining faces (which may include physical boundary faces). Let $c_e \in \mathbb{R}^M$ denote the volume coefficients of the element solution $u_h|_{K_e}(x) = \sum_{j=1}^M (c_e)_j \phi_j^{(\kappa)}(x)$.

Interior constraints. As in the single-tet case, the promoted Laplacian yields an active-row operator $L_{\text{int},e} \in \mathbb{R}^{M_{\text{int}} \times M}$ and a forcing vector $f_{\text{int},e} \in \mathbb{R}^{M_{\text{int}}}$ such that the strong interior equation is

$$(6.2) \quad L_{\text{int},e} c_e = -f_{\text{int},e}.$$

Boundary trace constraints (moment form). Choose a face test basis on each face (triangle polynomials of degree $\leq n$) and define, for any face set $B \subset \partial K_e$, the trace moment map

$$(T_{e,B})_{\ell j} := \int_B \psi_\ell(x) \phi_j^{(\kappa)}(x) \, dS.$$

We split the boundary into the interface face F and the remainder Γ_e , and write

$$(6.3) \quad T_{e,F} c_e = \hat{u}_F, \quad T_{e,\Gamma} c_e = \hat{u}_{\Gamma_e}.$$

Here $\hat{u}_F$ is the *unknown* interface trace coefficient vector (to be coupled to neighbors) and $\hat{u}_{\Gamma_e}$ is either known (Dirichlet data) or an auxiliary unknown depending on the global formulation.

Flux moments on F . Define the face flux moment map

$$(6.4) \quad (F_{e,F})_{\ell j} := \int_F \psi_\ell(x) (\nabla \phi_j^{(\kappa)}(x) \cdot n_e) \, dS,$$

where n_e is the outward unit normal of K_e on F . Then the interface flux coefficients induced by c_e are

$$(6.5) \quad \hat{q}_F^{(e)} = F_{e,F} c_e.$$

Nullspace parameterization of interior solutions. Equation (6.2) is underdetermined: it fixes c_e only up to a k -dimensional nullspace, where $k = M - M_{\text{int}} = (n+1)^2$ for the tetrahedron. We compute:

1. A particular solution $c_{p,e}$ satisfying

$$(6.6) \quad L_{\text{int},e} c_{p,e} = -f_{\text{int},e}.$$

2. A nullspace basis $N_e \in \mathbb{R}^{M \times k}$ satisfying

$$(6.7) \quad L_{\text{int},e} N_e = 0, \quad \text{rank}(N_e) = k.$$

Then every interior solution has the form

$$(6.8) \quad c_e = c_{p,e} + N_e y_e, \quad y_e \in \mathbb{R}^k.$$

Expressing y_e in terms of boundary trace data. Insert (6.8) into the boundary constraints (6.3):

$$(6.9) \quad T_{e,F}(c_{p,e} + N_e y_e) = \hat{u}_F,$$

$$(6.10) \quad T_{e,\Gamma}(c_{p,e} + N_e y_e) = \hat{u}_{\Gamma_e}.$$

Stacking these gives a linear system for y_e :

$$(6.11) \quad \underbrace{\begin{bmatrix} T_{e,F} N_e \\ T_{e,\Gamma} N_e \end{bmatrix}}_{=: \mathcal{T}_e \in \mathbb{R}^{N_b \times k}} y_e = \underbrace{\begin{bmatrix} \hat{u}_F \\ \hat{u}_{\Gamma_e} \end{bmatrix}}_{=: \hat{u}_{\partial K_e}} - \underbrace{\begin{bmatrix} T_{e,F} c_{p,e} \\ T_{e,\Gamma} c_{p,e} \end{bmatrix}}_{=: b_{p,e}}.$$

The matrix $\mathcal{T}_e$ is typically tall (many boundary moments) but has rank k for well-posed boundary data. To obtain a direct square solve we select k independent rows (pivoted QR on $\mathcal{T}_e^\top$) and solve the resulting $k \times k$ system for y_e .

Affine dependence on boundary traces.. Crucially, (6.11) is linear in the boundary traces:

$$y_e = \mathcal{T}_e^\dagger (\hat{u}_{\partial K_e} - b_{p,e}),$$

where $\mathcal{T}_e^\dagger$ denotes the (square, pivot-selected) inverse used in practice. Thus y_e depends *affinely* on $\hat{u}_{\partial K_e}$.

Eliminating c_e and deriving the DtN map. Substitute $c_e = c_{p,e} + N_e y_e$ into the flux relation (6.5):

$$(6.12) \quad \hat{q}_F^{(e)} = F_{e,F} c_{p,e} + F_{e,F} N_e y_e.$$

Now use (6.11) to express y_e in terms of $(\hat{u}_F, \hat{u}_{\Gamma_e})$. Because the solve for y_e is linear, we obtain an affine map

$$(6.13) \quad \hat{q}_F^{(e)} = S_{e,FF} \hat{u}_F + S_{e,F\Gamma} \hat{u}_{\Gamma_e} + \hat{q}_{F,f}^{(e)}.$$

Here:

- $S_{e,FF}$ maps interface Dirichlet trace to interface flux;
- $S_{e,F\Gamma}$ maps external-face Dirichlet data to interface flux;
- $\hat{q}_{F,f}^{(e)}$ is the forcing-induced flux contribution (including the contribution of $c_{p,e}$).

Recovering the one-face DtN form.. If $\hat{u}_{\Gamma_e}$ is prescribed (physical Dirichlet boundary), then the terms $S_{e,F\Gamma} \hat{u}_{\Gamma_e}$ and $\hat{q}_{F,f}^{(e)}$ are known and may be combined into a single forcing term:

$$(6.14) \quad \hat{q}_F^{(e)} = S_e \hat{u}_F + \hat{q}_{F,f}^{(e)},$$

which is the element DtN relation (6.1). If instead $\hat{u}_{\Gamma_e}$ contains additional interface unknowns (e.g. when K_e has multiple shared faces), then (6.13) provides the corresponding block structure needed for multi-face coupling and for hierarchical merging.

6.6. Two-tet coupling: flux continuity. On the shared face, continuity of normal flux with opposite normals gives

$$\hat{q}_F^{(1)} + \hat{q}_F^{(2)} = 0.$$

Substituting (6.1) yields a square interface system for $\hat{u}_F$:

$$(6.15) \quad (S_1 + S_2) \hat{u}_F = -(\hat{q}_{F,f}^{(1)} + \hat{q}_{F,f}^{(2)}).$$

Once $\hat{u}_F$ is solved, recover each c_e locally (downward pass / back-substitution).

7. From two tetrahedra to a mesh: adjacency-graph and HPS viewpoint.

7.1. Dual graph and separators. Let $\mathcal{T}$ be a tetrahedral mesh. Consider the dual graph whose nodes are tetrahedra and whose edges connect tetrahedra sharing a face. A hierarchical solver (HPS/nested-dissection style) recursively partitions the dual graph into subgraphs, producing separators that correspond to collections of faces on subdomain boundaries.

7.2. Subdomain DtN operator. For any subdomain Ω_τ (a union of tetrahedra), define its boundary interface Γ_τ as the union of faces on the boundary of the subdomain. A subdomain DtN map is

$$\hat{q}_{\Gamma_\tau} = S_\tau \hat{u}_{\Gamma_\tau} + \hat{q}_{\Gamma_\tau,f},$$

where $\hat{u}_{\Gamma_\tau}$ are trace coefficients on the subdomain boundary and $\hat{q}_{\Gamma_\tau}$ are the associated normal flux coefficients.

7.3. Upward pass (merge). Suppose a parent subdomain τ is formed by merging two children τ_L and τ_R that share an internal interface Γ_{int} . Each child provides a DtN map on its boundary:

$$\hat{q}_{\Gamma_L} = S_L \hat{u}_{\Gamma_L} + \hat{q}_{L,f}, \quad \hat{q}_{\Gamma_R} = S_R \hat{u}_{\Gamma_R} + \hat{q}_{R,f}.$$

The internal interface traces $\hat{u}_{\Gamma_{\text{int}}}$ are unknown and must satisfy flux continuity when the children are glued. Eliminating $\hat{u}_{\Gamma_{\text{int}}}$ produces a new DtN map S_τ on the *external* boundary Γ_τ of the merged domain. Algebraically, this is a Schur-complement elimination on interface blocks, analogous to (6.15) but at a larger scale.

7.4. Root solve. At the root of the hierarchy, $\Gamma_{\text{root}} = \partial\Omega$. Impose boundary data (Dirichlet traces) to solve for the remaining interface unknowns.

7.5. Downward pass (back-substitution). Once parent boundary traces are known, recursively recover child traces on internal interfaces and then recover each tetrahedron’s volume coefficients using the stored local elimination data (nullspace completion / element solves).

Complexity remarks.. Without compression, separator sizes govern complexity (nested dissection). With compression (e.g. low-rank structure in interface-to-interface maps), HPS-type methods can achieve near-linear complexity for elliptic problems. In the present work, the key enabling property is the availability of sparse/banded modal differentiation and promotion operators on each tetrahedron, allowing high-order local solves and efficient DtN construction.

8. Image Compression via Jacobi Polynomial Expansion. An image (i.e. with file format .jpg, .png, etc.) can be thought of as an integer-valued surface (or multiple surfaces in the case of multi-channel images) on a rectangular domain. We can easily tessellate this domain with triangles equipped with the quadrature nodes derived in the previous section. The idea is to approximate the subsurfaces on each triangle with a Jacobi polynomial expansion. Areas where the image is highly varying will require more triangles or higher order expansion (i.e. h vs p refinement). As of now, we can compute expansion coefficients on the reference triangle. Here, we first derive a method for computing such expansions on a general triangle. For a given function $f(x, y)$, with $(x, y) \in \mathcal{T}^2$ points in a general triangle

Let $X = \{x_1, x_2, \dots, x_k\}$ be a set of k signed integers, where each x_i is represented using n bits. Let $y = \sum_{i=1}^k x_i$ be the sum of all elements in X , which is representable using $m \geq n + 1$ bits without overflow (i.e., $-2^{m-1} \leq y \leq 2^{m-1} - 1$). Then, there exists an ordering of summation for X that avoids transient overflow when using an m -bit accumulator.

Base case: Suppose $k = 2$. The set $X = \{x_1, x_2\}$ is comprised of n -bit numbers, and representing the sum $x_1 + x_2$ can require at most $n + 1$ bits. Since $m \geq n + 1$, the sum does not overflow m -bits. By the commutative property of binary addition, the sum $x_2 + x_1$ will likewise not overflow m -bits. Hence, any ordering of X avoids transient overflow, and so the theorem holds for $k = 2$.

Inductive hypothesis: Consider an integer $l \geq 2$. Assume that the theorem holds $\forall k \leq l$. That is, there exists an ordering of the set $X = \{x_1, \dots, x_l\}$ such that the sum of elements of X w.r.t said ordering avoids transient overflow. Denote this ordering by the index set α_l , and the so-ordered set by X_{α_l} .

Inductive step: Suppose that $k = l + 1$. Consider the set $X = X_{\alpha_l} \cup x_k$. Assume that the sum of all elements of X is representable by an m -bit signed integer y . We know that

$$y = \sum_{i=1}^k x_i = x_k + \sum_{i \in \alpha_l} x_i.$$

By the inductive hypothesis, the second term in the sum avoids transient overflow. Denote this second term by $\hat{y} = \sum_{i \in \alpha_l} x_i$. Since $\hat{y}$ is an m -bit signed integer, and x_k is an n -bit signed integer with $n \leq m - 1$, the sum $x_k + \hat{y}$ is represented by at most m bits. Therefore, a feasible ordering to avoid transient overflow is $\alpha_k = \{\alpha_l, k\}$.

Thus, by induction, our theorem must hold for any $k \geq 2$.